KH정보교육원

공공데이터 활용하기

CONTENTS

Chapter 1. 공공데이터 이해하기

1-1. 공공데이터 개요 4

1-2. 공공데이터 개방 7

Chapter 2. 공공데이터 분석하기

2-1. 서비스 신청하기 13

2-2. 개발문서 분석하기 16

Chapter 3. 공공데이터 활용하기

3-1. 분석한 공공데이터 웹애플리케이션 적용하기 21

3-2. 지오코딩 활용한 지도 API 활용하기 36

3-3. 공공데이터 활용사례 연구를 통한 웹애플리케이션 적용하기 48

Chapter 1. 공공데이터 이해하기

1-1. 공공데이터 개요

1 공공데이터의 의미

공공 데이터(영어: open data)는 데이터베이스, 전자화된 파일 등 공공기관이 법령 등에서 정하는 목적을 위하여 생성 또는 취득하여 관리하고 있는 광(光) 또는 전자적 방식으로 처리된 자료 또는 정보로서 다음과 같은 것을 지칭한다. 첫째, 「전자정부법」에 따른 행정정보, 둘째, 「국가정보화 기본법」에 따른 정보 중 공공기관이 생산한 정보, 셋째, 「공공기록물 관리에 관한 법률」에 따른 전자기록물 중 대통령령으로 정하는 전자기록물, 그 밖에 대통령령으로 정하는 자료 또는 정보가 있다. 따라서, 공공데이터는 개별 공공기관이 일상적 업무수행의 결과물로 생성 또는 수집·취득한 다양한 형태(텍스트, 수치, 이미지, 동영상, 오디오 등)의 모든 자료 또는 정보를 대상으로 하며, "기계 판독이 가능한 형태"로 "제공"이되어야 한다. 여기서 "기계 판독이 가능한 형태"란 소프트웨어로 데이터의 개별 내용 또는 내부 구조를 확인하거나 수정, 변환, 추출 등 가공할 수 있는 상태를 말하며, "제공" 이란 공공기관이 이용자로 하여금 기계 판독이 가능한 형태의 공공데이터에 접근할 수 있게 하거나 이를 다양한 방식으로 전달하는 것을 말한다.

정부는 보유한 공공 정보를 적극적으로 개방하여 국민과 공유함으로써 소통과 협력을 확대하기 위해 공공데이터 정책을 추진하게 되었고, 2013년 7월 공공데이터의 제공 및 이용활성화에 관한 법률(약칭 : 공공데이터법)을 제정하고 10월부터 시행 되었다.

공공데이터 제공 및 이용 활성화에 관한 정책은 '행정안전부'가 총괄하며, 이 법률에 근거하여 2013년 11월, 정부나 공공기관이 보유한 공공데이터를 민간에서 쉽게 활용할 수 있도록 지원하기 위하

여 한국정보화진흥원에 공공데이터활용지원센터'를 설치 하였다.[3] 각 공공기관은 공공데이터제공책임관 및 실무자를 지정하고 관련 업무를 지원한다.[4] 또한 정부는 공공데이터 관련 정책을 심의, 조정 및 그 추진사항을 점검하기 위하여 국무총리실 산하에 공공데이터전략위원회[5]를 설립하였고, 공공기관의 공공데이터 제공거부 또는 중단 문제 발생시 해결을 위한 공공데이터제공분쟁조정위원회[6]를 설립하여 공공데이터 활성화 정책을 추진하고 있다.

2 공공데이터 개방

공공데이터 개방이란 정부, 지방자치단체 및 공공기관이 보유·관리하는 공공데이터를 이용자가 자유롭게 활용할 수 있도록 다양한 형태로 개방·제공하는 것이다. 대한민국 정부, 지방자치단체 및 공공기관은 공공데이터를 이용자가 자유롭게 활용할 수 있도록 다양한 형태로 개방·제공하고 있다. 단, 개인정보보호, 제3자의 권리(저작권 등)가 포함된 정보, 국가 안보와 질서에 관련한 정보 등 제외된다.

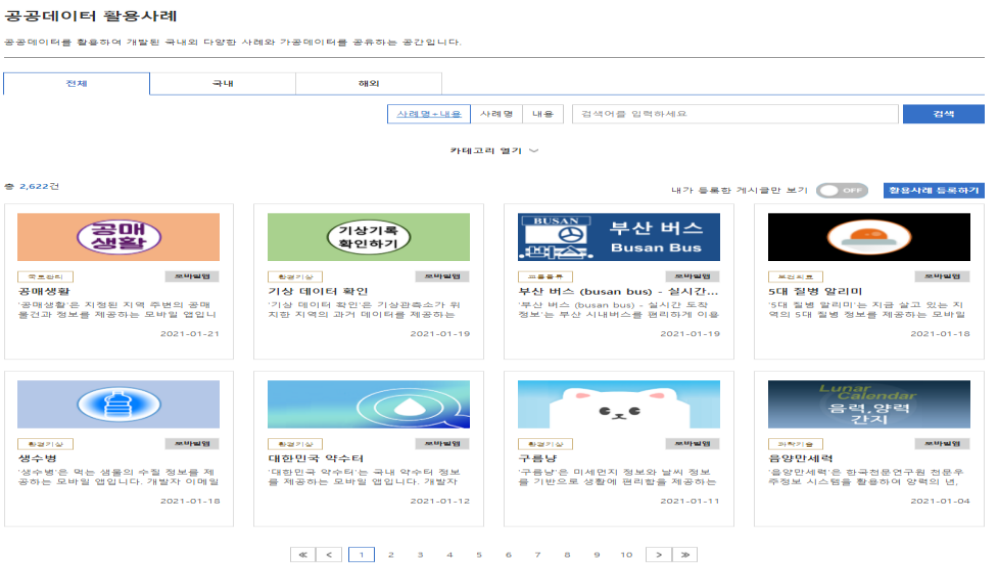
공공 데이터는 누구나 자유롭게 이용할 수 있으며 공공데이터포털을 통해 국민이 쉽고 편리하게 공공데이터를 이용할 수 있도록 파일데이터, 오픈API, 시각화 등 다양한 방식으로 제공하고 있으며, 누구라도 쉽고 편리한 검색을 통해 원하는 공공데이터를 빠르고 정확하게 찾을 수 있다. 공공데이터 개방관련 공공데이터포털에 대한 내용은 chapter2에서 자세히 다뤄보도록 하자.

3 공공데이터 활용

정부는 공공데이터 이용에 대한 국민의 인식을 높이고 이용 활성화를 촉진하기 위하여, 공공데이터 활용 기업의 창업에서부터 성장까지 전과정을 지원한다. 공공데이터 활용 창업 공간인 오픈스퀘어-D와 공공데이

터 활용 창업경진대회가 그 대표적인 예이다. 오픈스퀘어-D는 공공데이터에 대한 창의적인 아이디어를 가진 사람들이 경험과 기술을 교류하고, 사업화.창업으로 성장할 수 있도록 종합적으로 지원하는 열린공간이다. 오픈스퀘어-D는 창업자들을 위한 입주공간, 아이디어 교류 및 발전을 위한 협업공간, 서비스 개발에 필요한 테스트 및 데이터 분석을 위한 테스트 공간으로 구성되어 있다. 창업단계별 교육, 맞춤형 컨설팅, 투자유치 및 네트워킹 등 다양한 프로그램이 제공된다.' 공공데이터 활용 창업경진대회'는 공공데이터를 활용한 아이디어 또는 웹(앱) 제품 및 서비스 개발 아이템을 발굴하는 대회이다. 민간의 창의적인 아이디어를 공공데이터와 융합하여 새로운 비즈니스 모델을 발굴해 공공데이터 기반의 창업촉진 및 창업기업 육성을 목적으로 매년 1회 개최된다. 국민 누구나 참여가 가능하며, 수상자에게는 시상금뿐만 아니라 제품 및 서비스 홍보, 사무공간 지원, 투자유치, 창업컨설팅 등 다양한 혜택이 제공된다.

공공데이터활용 관련 내용은 공공데이터 포털에서 확인할 수 있으며 어떤 공공데이터가 어떤 애플리케이션에 적용됐는지 확인할 수 있도록 아래와 같이 제공하고 있다. (https://www.data.go.kr/tcs/puc/selectPublicUseCaseListView.do)



1-2 공공데이터 개방

1 의미

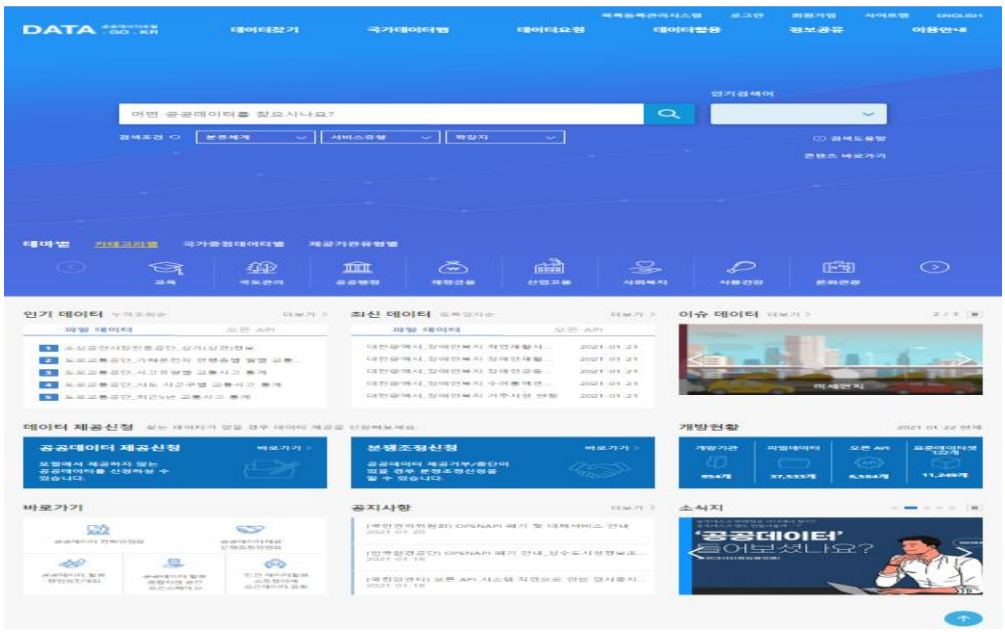
정부, 지방자치단체 및 공공기관이 보유, 관리하고 있는 공공데이터를 다양한 형태로 개방, 제공하는 것을 말한다. 현재 정부는 공공데이터 포털 웹애플리케이션을 운영 중하고 있으며 사용자는 웹애플리케이션에 접속하여 편리하게 데이터를 확인하고 활용할 수 가 있다.

2 공공데이터포털

정부에서 운영하고 있는 공공데이터를 제공하는 웹애플리케이션으로 누구나 자유롭게 공공데이터를 확인하고 관련 내용을 활용하여 애플리케이션을 만들 수 있게 도와준다.

2.1 공공데이터 포털 둘러보기

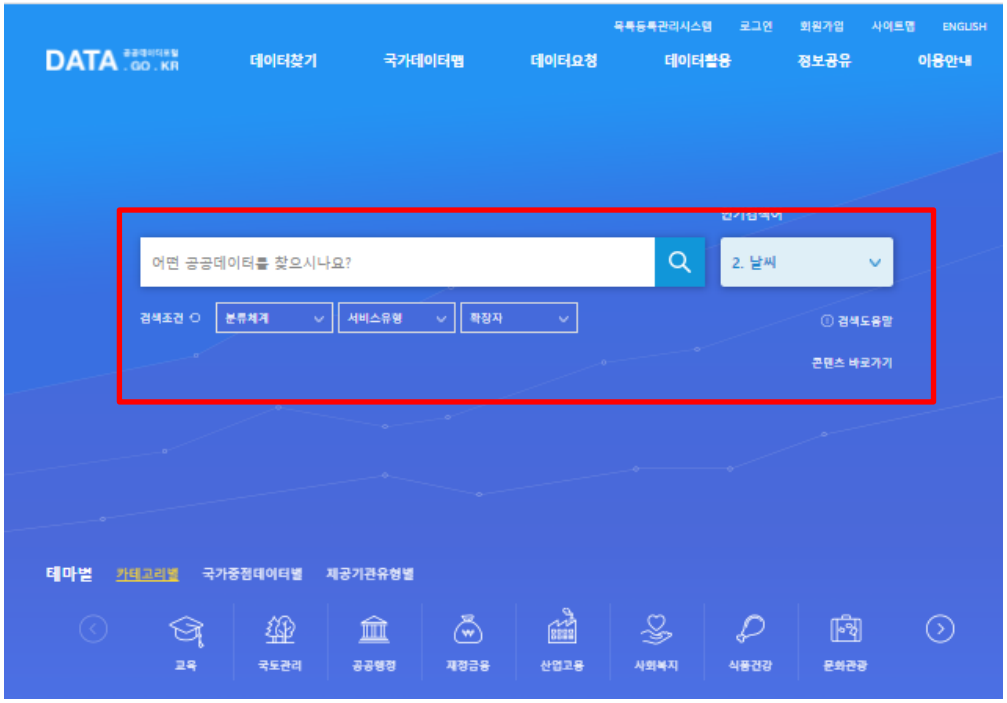
https://www.data.go.kr/ 접속하기



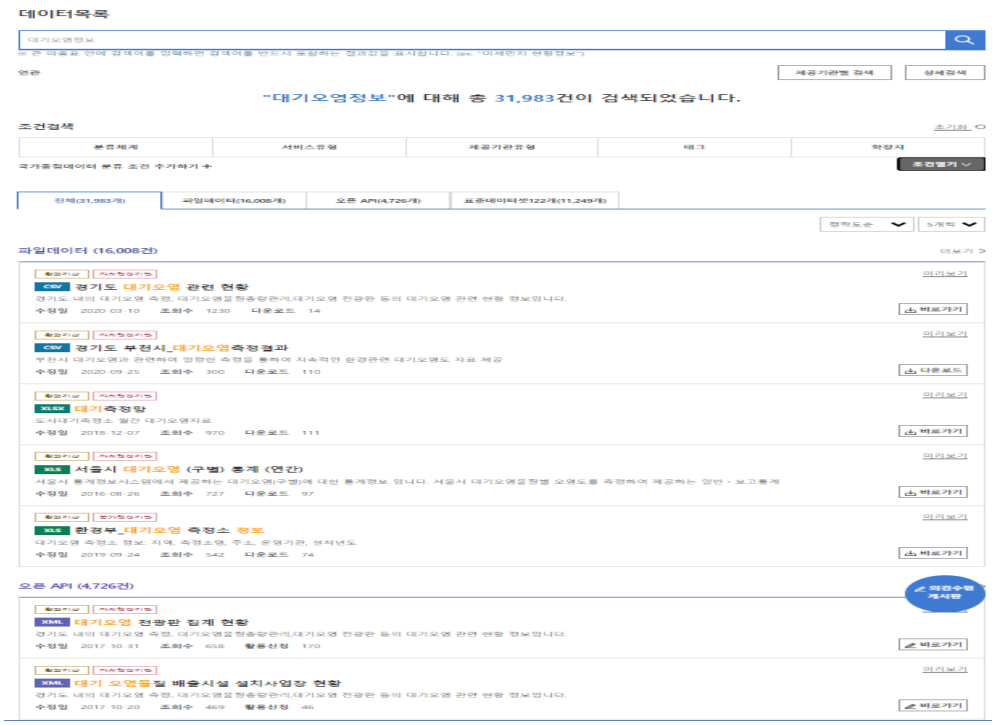
메인화면은 중간부분에 통합조회를 하는조회창이 있고, 각종 공공데이터를 간편하게 조회할 수 있도록 카테고리별, 인기데이터, 최신데이터 등을 리스트업 해놨으며, 각종 정보를 확인요청을 할 수 있게 구성을 해놨다.

2.2 공공데이터 조회하기

공공테이터 포털에서는 다양한 방법으로 데이터를 조회할 수 있다. 먼저 통합검색을 이용해 보도록 하자.



페이지의 상단 중간에 검색창이 데이터를 통합검색할 수 있는 창이다. Keyword를 입력하면 keyword와 관련된 데이터가 조회되서 나오게 된다. '대기오염정보'를 입력해보자.

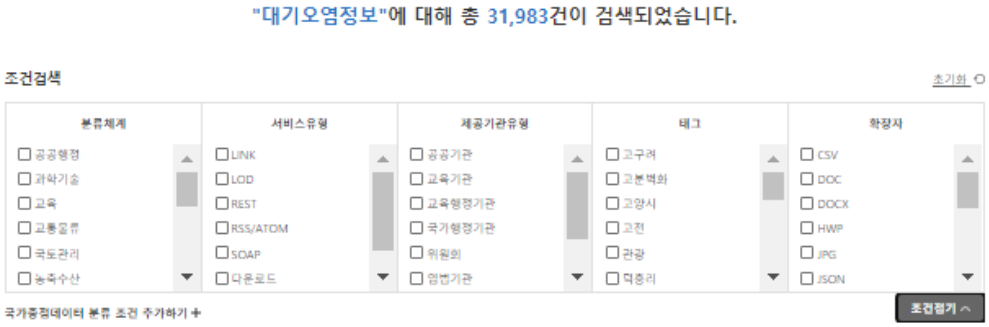


대기오염과 관련된 데이터 들이 조회가 되며 제공하는 방식에 따라 파일 데이터, 오픈API, 표준데이터셋으로 구분되어있는 것을 확인할 수 있다.

- 파일데이터방식 : CSV, XML, exls형태의 파일로 데이터를 제공하는 것을 말한다. 이 방식의 데이터는 사용자가 직접 파일을 다운로드 하여 사용할 수 있다.

- 오픈API방식 : HttpRequest요청을 받으면 서버가 데이터를 보내주는 방식으로 요청주소, 파라미터, 반환데이터 등이 정해져있으며 이를 이용하기 위해서는 인증키를 발급받아야 한다. 웹 애플리케이션을 만들때는 이 방식이 파일다운로드 하는 것보다 더 나아보인다.

추가적으로 조건열기를 선택하면 아래 그림과 같이 세부적인 항목이 나오고 조회도 가능하다.



데이터 상세화면 보기

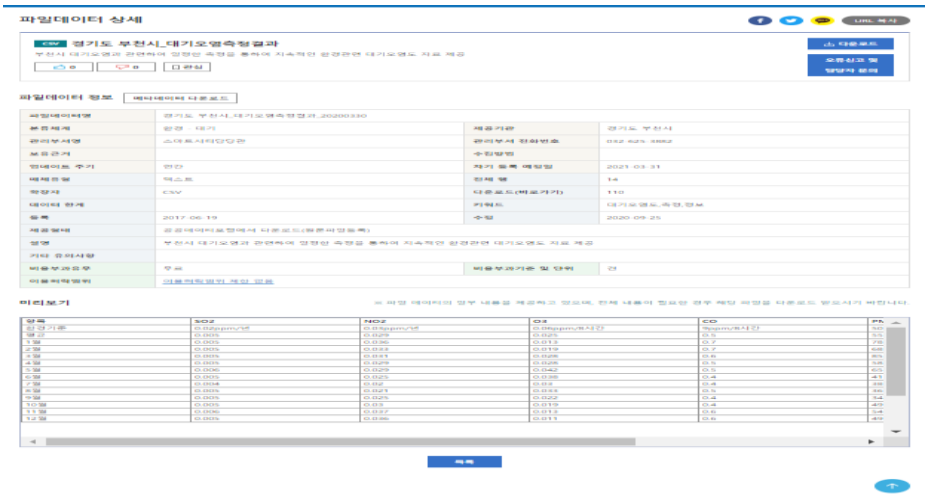
검색화면에 나온 데이터를 클릭하면 상세화면으로 들어가게 되는데 상세화면은 파일데이터방식과 오픈API방식이 약간 상이하다. 먼저 파일데이터방식의 상세화면을 보자.

검색내용중 '경기도 부천시_대기오염 측정결과' 데이터를 클릭해보자.

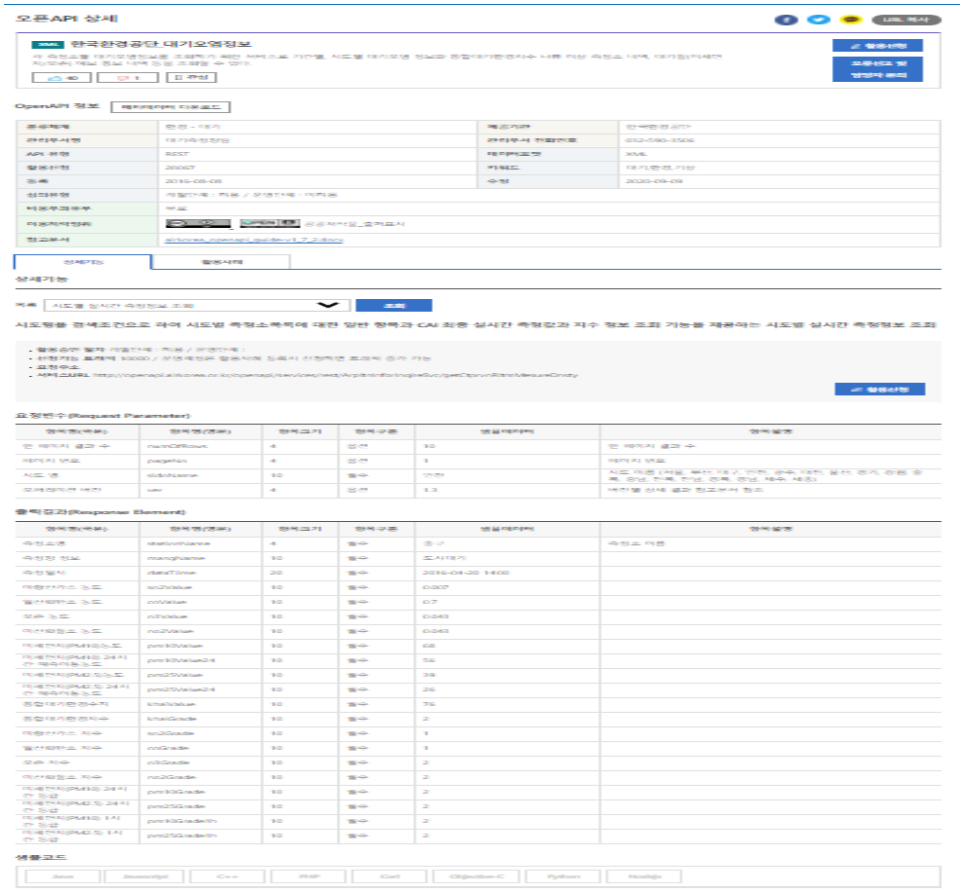
*파일데이터방식은 페이지에서 바로다운받을 수 있는 데이터도 있고 링크로 연결되어있는 데이터도 있다.

상세화면 왼쪽 맨위에 다운로드 버튼을 이용하면 파일을 다운로드할 수 있고 메타데이터로 파일의 확장자, 제공자 분류 등을 확인할 수 있다.

그리고 아래쪽에는 미리보기가 있어 어떤 데이터가 어떻게 들어가 있는지 한눈에 볼수가 있다.



이번에는 오픈 API 상세화면을 확인해보자. 오픈 API 도 파일을 제공하는 방식이 있는데 xml, json 방식으로 제공을 한다. 검색결과 탭에서 오픈 API 를 선택하고 '한국환경공단_대기오염정보'를 클릭해보자.

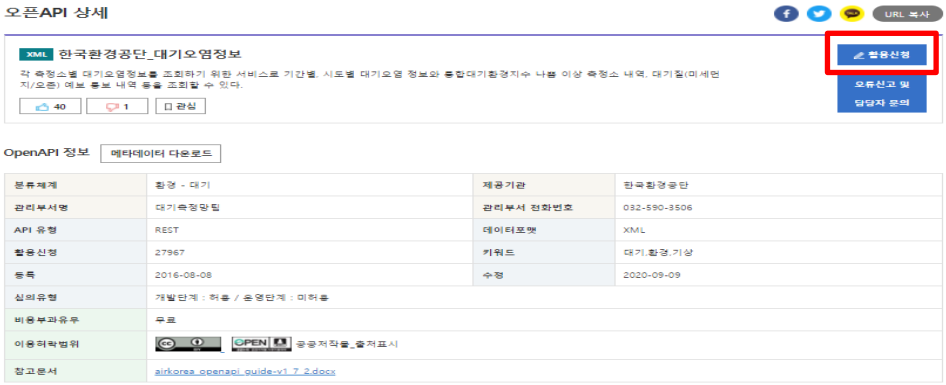


오픈 API 방식을 활용신청을 해서 인증키를 받아야 사용이 가능하다. 요청할 주소, 요청변수, 출력결과를 확인 할 수 있다. 그리고 데이터 이용 가이드라고 할 수 있는 참고문서가 있다 참고문서에는 상세한 정보들이 있기 때문에 반드시 다운받고 확인해야 한다.

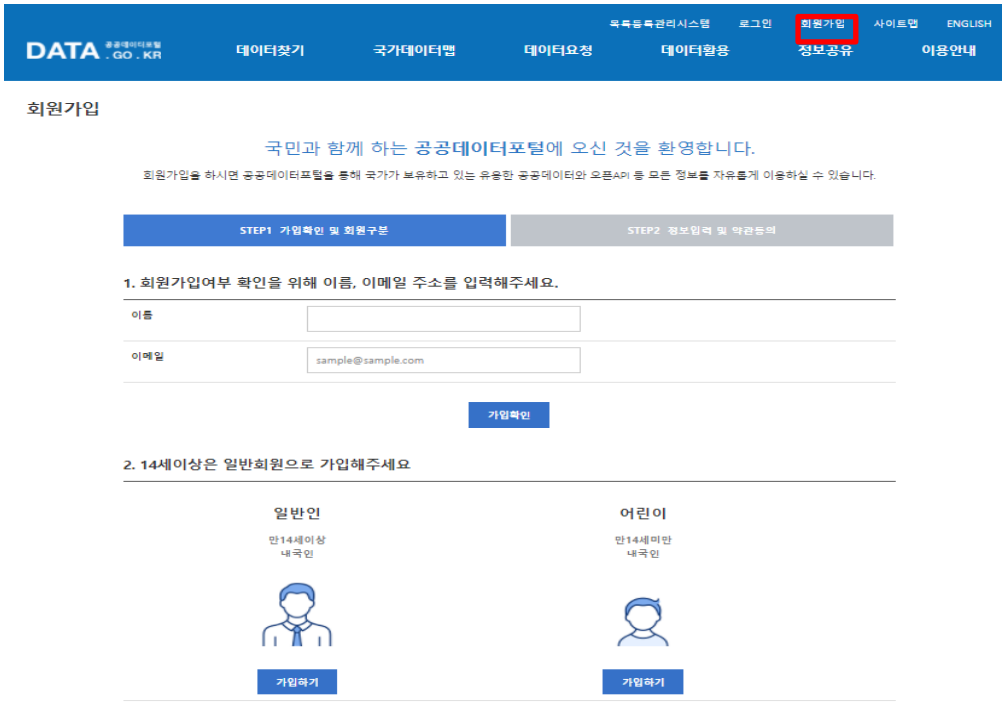
Chapter 2. 공공데이터 분석하기

2-1 서비스 신청하기

오픈 API를 사용하려면 활용신청 후 인증키를 부여받은 후 사용이 가능하다. 상세화면에서 활용신청 버튼을 클릭한다.



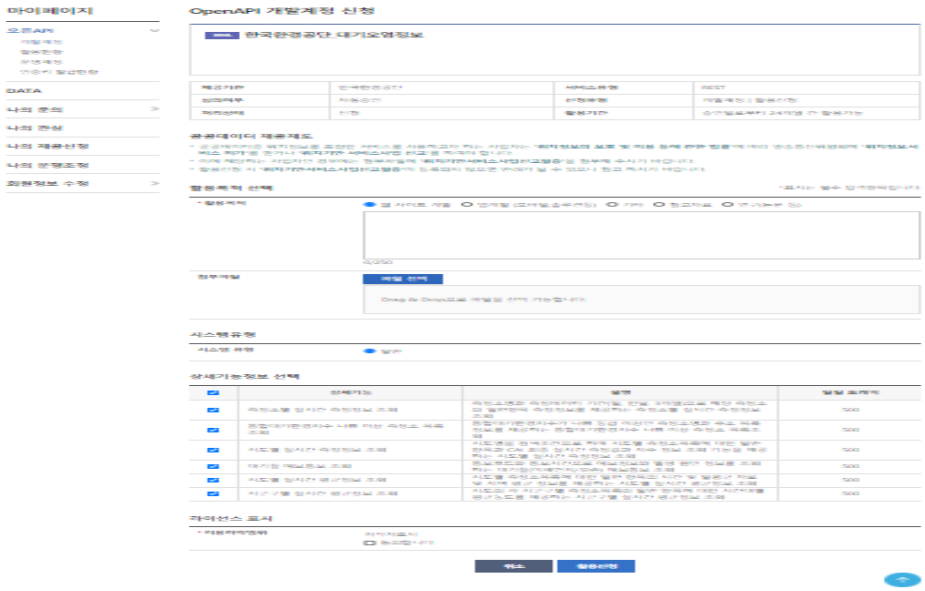
활용신청을 하기 위해서는 회원가입이 필요하다.



이름과 이메일을 입력하고 가입확인 후 가입을 진행한다. 이메일 인증과 핸드폰인증이 있으니 이메일을 반드시 본인이 쓰고 있는 메일주소로 작성해야한다. 세부정보를 입력하고 가입을 완료한다.



세부정보를 입력하고 가입을 완료 후 로그인하고 활용신청을 다시 진행한다. 활용신청을 하면 활용 목적을 작성하는데 참고자료선택 "직업훈련교육 참고자료"로 작성하자.



활용신청을 하고 나면 마이페이지에서 신청한 데이터에 대한 정보가 확인이 가능하다. 신청후 발급된 인증키로 데이터를 이용하려면 1~2 시간이 소요된다.



승인된 데이터의 상세내용에 들어가면 승인받은 공공데이터에 대한 상세정보와 인증키를 확인할 수 있다. 대기오염정보 중 여러가지 조회기능을 제공하는것도 확인할 수 있음 이중 내가 원하는 데이터를 활용하면 된다. 서비스 정보부분을 보면 참고 문서가 있는데 참고문서에는 데이터활용에 대한 더 상세한 정보가 있기 때문에 공공데이터를 이용할때는 이 문서를 이용하는 것이 좋다.

2-2 개발문서 분석하기

공공데이터에서 반환되는 데이터에 대한 정보를 분석하는 과정으로 데이터를 로 서버에서 해당를 서버에서 받아서 클라이언트에게 응답을 해주기 위해 필요한 데이터로 정제화하는 작업이라고 보면 된다.

반환되어오는 데이터의 상세 정보를 확인하려면 참고문서를 확인하면 된다. 참고문서를 열어보면 각 서비스에 대한 상세한 정보가 나온다.

대기오염정보 조회 서비스를 웹애플리케이션에 제공할 예정으로 관련 내용을 에 대해 살펴보자.

▪ 가. 서비스 개요

서비스 정보	서비스 ID	SC-OA-09-02	
	서비스명(국문)	대기오염정보 조회 서비스	
	서비스명(영문)	ArpltnInforInqireSvc	
	서비스 설명	각 측정소별 대기오염정보를 조회하기 위한 서비스로 기간별, 시도별 대기오염 정보와 통합대기환경지수 나쁨 이상 측정소 내역, 대기질(미세먼지/오존) 예보 통보 내역 등을 조회할 수 있다.	
서비스 보안	서비스 인증/권한	[O] 서비스 Key [ ] 인증서 (GPKI) [ ] Basic (ID/PW) [ ] 없음	[ ]WS-Security
	메시지 레벨 암호화	[ ] 전자서명 [ ] 암호화 [O] 없음	
	전송 레벨 암호화	[ ] SSL [O] 없음	
적용 기술 수준	인터페이스 표준	[ ] SOAP 1.2(RPC-Encoded, Document Literal, Document Literal Wrapped) [O] REST (GET) [ ] RSS 1.0 [ ] RSS 2.0 [ ] Atom 1.0 [ ] 기타	
	교환 데이터 표준	[O] XML [O] JSON [ ] MIME [ ] MTOM	
서비스 URL	개발환경	http://openapi.airkorea.or.kr/openapi/services/rest/ArpltnInforInqireSvc	
	운영환경	http://openapi.airkorea.or.kr/openapi/services/rest/ArpltnInforInqireSvc	
서비스 배포 정보	서비스 버전	1.1	
	서비스 시작일	2013-12-05 / 배포 일자 / 2013-12-05	
	서비스 이력	(갱신) 2016-08-08 : 시도별 실시간 평균정보 조회 및 시군구별 실시간 평균정보 조회 오퍼레이션 추가	
메시지 교환 유형		[O] Request-Response [ ] Publish-Subscribe [ ] Fire-and-Forgot [ ] Notification	
메시지 로깅 수준		성공 / [O] Header [ ] Body / 실패 / [O] Header [O} Body	
사용 제약 사항 (비고)		N/A	
데이터 갱신주기		- 실시간 정보 : 10분(매 시간 시간자료 갱신은 20분 전후로 반영됨) - 대기질 예보 정보 : 매 시간 22분, 57분	

※ JSON 방식 호출 방법 : 호출 시 다음의 파라미터(&_returnType=json)를 추가하여 호출

먼저 서비스 개요가 나오는데 서비스에 대한 주요 정보들이 나온다. 여기서 중요한 것은 서비스에 대한 주요 내용, 데이터 반환 방식(JSON, XML 등), 데이터 갱신주기 등을 확인해 보면 된다. 관련정보를 숙지해서 서비스는 제공하는 시기(갱시주기)와 어떤 정보를 제공할 수 있는지 파악할 수 있다.

오퍼레이션 목록은 말그대로 조회되는 서비스에 대한 이름을 의미한다고 보면 된다. *상세화면에서 미리보기 샘플이 있었던 내용들이다.

▪ 나. 오퍼레이션 목록

일련번호	서비스명(국문)	오퍼레이션명(영문)	오퍼레이션명(국문)	메시지명(영문)
1	대기오염 정보 조회 서비스	getMsrstnAcctoRltmMesureDnsty	측정소별 실시간 측정정보 조회	N/A
2		getUnityAirEnvrnIdexSnstveAboveMsrstnList	통합대기환경지수 나쁨 이상 측정소 목록조회	N/A
3		getCtprvnRltmMesureDnsty	시도별 실시간 측정정보 조회	N/A
4		getMinuDustFrcstDspth	대기질 예보통보 조회	N/A
5		getCtprvnMesureLIst	시도별 실시간 평균정보 조회	N/A
6		getCtprvnMesureSidoLIst	시군구별 실시간 평균정보 조회	N/A

이 오퍼레이션 중 서비스할 오퍼레이션을 선택하여 분석하면 된다. 이중 시도별 실시간 측정정보조회 정보확인 하고 제공해보자. 아래와 그림과 같이 각 서비스별 오퍼레이션 명세가 있는데 이것을 확인하여 데이터를 확인하고 분석할 수 있다.

▪ (3) 시도별 실시간 측정정보 조회 오퍼레이션 명세

오퍼레이션 정보	오퍼레이션 번호	3	오퍼레이션명(국문)	시도별 실시간 측정정보 조회
	오퍼레이션 유형	조회	오퍼레이션명(영문)	getCtprvnRltmMesureDnsty
	오퍼레이션 설명	시도명을 검색조건으로 하여 시도별 측정소목록에 대한 일반 항목과 CAI 최종 실시간 측정값과 지수 정보 조회 기능을 제공하는 시도별 실시간 측정정보 조회		
	URL end-point	http://openapi.airkorea.or.kr/openapi/services/rest/ArpltnInforInqireSvc/getCtprvnRltmMesureDnsty		
	최대 메시지 사이즈	[1000K bytes]		
	평균 응답 시간	[500 ms]	초당 최대 트랜잭션	[30 tps]

▪ 요청 메시지 명세

항목명(영문)	항목명(국문)	항목크기	항목구분	샘플데이터	항목설명
ServiceKey	서비스키	-	1	aAb12....%30	서비스키
numOfRows	한 페이지 결과 수	4	0	10	한 페이지 결과 수
pageNo	페이지 번호	4	0	1	페이지 번호
sidoName	시도 명	10	1	서울	시도 이름 (서울, 부산, 대구, 인천, 광주, 대전, 울산, 경기, 강원, 충북, 충남, 전북, 전남, 경북, 경남, 제주, 세종)
ver	오퍼레이션 버전	4	0	1.0	버전별 상세 결과 아래쪽 참고

서비스별 오퍼레이션 명세가 있는데 이것을 확인하여 데이터를 확인하고 분석할 수 있다. 오퍼레이션 정보에서는 요청을 하기 위한 URL 주소, 요청에 대한 서비스 주요내용이 있으며 요청하기 위한 Parameter(요청메세지 명세)들이 명시가 되어있다.

요청주소 : http://openapi.airkorea□.or.kr/openapi/services/rest/ArpltnInforInqireSvc/getCtprvnRltmMesureDnsty

ServiceKey : 활용신청을 했을 때 부여 받은 인증키을 값으로 넣어 보내면 된다. 필수값

numOfRows : 한페이지 결과값 선택값 기본 10

pageNo : 페이지번호 선택값 기본 1

sidoName : 조회한 지역에 대한 이름, 항목설명에 있는 지명으로 작성 필수값

* 파라미터에는 필수적으로 넣어야하는 값과 선택하는 값이 있다. 선택은 설정하지않으면 기본값으로 설정됨

필수적인 값을 파라미터값에 넣고 HttpRequest를 보내면 응답하는 메시지 명세도 상세하게 나열되어있다. 데이터 분석을 위해서는 이 명세서를 잘 분석하여 DB테이블 및 DTO(VO)를 설계할 수 있다. 간단하게 보면 응답메세지는 요청시에 넘어오는 데이터들에 대한 key값과 value값을 의미하고, 데이터는 JSON방식과 XML방식으로 데이터가 반환된다. JSON에서는 항목명(영문)은 key값이 되고 value는 샘플데이터에서 있는 내용들이다. XML에서는 items태그안에 항목명(영문)이 자식 태그가되고 각 자식태그안의 값이 샘플데이터에 있는 값이 된다.

항목별 설명을 보고 웹애플리케이션 구현에 필요한 내용을 추려내면 되겠다. 추려진 데이터의 key값과 샘플데이터의 내용을 확인해서 데이터를 받을 DTO(VO)클래스를 설계할 때 참조하면 되겠다. 아래 내용을 분석하여 DTO(VO)를 설계해보자.

▪ 응답 메시지 명세

항목명(영문)	항목명(국문)	항목크기	항목구분	샘플데이터	항목설명
resultCode	결과코드	2	1	00	결과코드
resultMsg	결과메세지	50	1	[illegible]	결과메세지
numOfRows	한 페이지 결과 수	4	1	10	한 페이지 결과 수
pageNo	페이지 번호	4	1	1	페이지 번호
totalCount	전체 결과 수	4	1	40	전체 결과 수
items	목록	-	0..n	-	목록
stationName	측정소명	30	1	중구	측정소명
mangName	측정망 정보	10	1	도시대기	측정망 정보 (국가배경, 교외대기, 도시대기, 도로변대기)
dataTime	측정일시	20	1	2016-04-20 14:00	오염도 측정 연-월-일 시간 : 분
so2Value	아황산가스 농도	10	1	0.007	아황산가스 농도(단위 : ppm)
coValue	일산화탄소 농도	10	1	0.7	일산화탄소 농도(단위 : ppm)
o3Value	오존 농도	10	1	0.043	오존 농도(단위 : ppm)
no2Value	이산화질소 농도	10	1	0.043	이산화질소 농도(단위 : ppm)
pm10Value	미세먼지(PM_{10}) 농도	10	1	68	미세먼지(PM_{10}) 농도 (단위 : ㎍/㎥)
pm10Value24	미세먼지(PM_{10}) 24시간예측이동농도	10	1	56	미세먼지(PM_{10}) 24시간예측 이동농도(단위 : ㎍/㎥)
pm25Value	미세먼지($PM_{2.5}$) 농도	10	1	39	미세먼지($PM_{2.5}$) 농도 (단위 : ㎍/㎥)
pm25Value24	미세먼지($PM_{2.5}$) 24시간예측이동농도	10	1	26	미세먼지($PM_{2.5}$) 24시간예측 이동농도(단위 : ㎍/㎥)
khaiValue	통합대기환경수치	10	1	76	통합대기환경수치
khaiGrade	통합대기환경지수	10	1	2	통합대기환경지수
so2Grade	아황산가스 지수	10	1	1	아황산가스 지수
coGrade	일산화탄소 지수	10	1	1	일산화탄소 지수
o3Grade	오존 지수	10	1	2	오존 지수
no2Grade	이산화질소 지수	10	1	2	이산화질소 지수
pm10Grade	미세먼지(PM_{10}) 24시간등급자료	10	1	2	미세먼지(PM_{10}) 24시간 등급자료
pm25Grade	미세먼지($PM_{2.5}$) 24시간등급자료	10	1	2	미세먼지($PM_{2.5}$) 24시간 등급자료
pm10Grade1h	미세먼지(PM_{10}) 1시간등급자료	10	1	2	미세먼지(PM_{10}) 1시간 등급자료
pm25Grade1h	미세먼지($PM_{2.5}$) 1시간등급자료	10	1	2	미세먼지($PM_{2.5}$) 1시간 등급자료

※ 항목구분 : 필수(1), 옵션(0), 1건 이상 복수건(1..n), 0건 또는 복수건(0..n)

※ 항목별 Grade 값의 의미

- 적용 항목명 : khaiGrade, so2Grade, coGrade, o3Grade, no2Grade, pm10Grade, pm25Grade, pm25Grade1h, pm25Grade1h

등급	좋음	보통	나쁨	매우나쁨
Grade 값	1	2	3	4

위 자료의 내용을 확인하고 서비스 제공에 필요한 데이터를 추릴 수 있고 샘플데이터의 내용을 보고 DTO(VO)의 필드의 변수명 자료형을 결정할 수 있다.

Chapter 3. 공공데이터 활용하기

3-1 분석한 공공데이터 웹애플리케이션 적용하기

전체 데이터를 모두 가져와 활용한다고 했을 때 클래스를 설계하면 다음과 같이 설계할 수 있을 것이다.

```
public class AirDto {

	private String stationName;
	private String mangName;
	private String dataTime;
	private int khaiGrade;
	private int khaiValue;
	private int pm10Grade;
	private int pm10Grade1h;
	private int pm10Value;
	private int pm10Value24;
	private int pm25Grade;
	private int pm25Grade1h;
	private int pm25Value;
	private int pm25Value24;
	private int coGrade;
	private int no2Grade;
	private int o3Grade;
	private int so2Grade;
	private double coValue;
	private double no2Value;
	private double o3Value;
	private double so2Value;
```

샘플자료를 보고 문자열형식이면 String, 정수 int, 실수형이면 double 로 설계하면 된다. 여기서 문제되는 것이 Date 를 표현하는 dataTime 인데 서버상에서 요청과 응답을 하게 되면 문자열로 처리가 되기 때문에 다른 파싱(Date 로 변환)하는 작업없이 그냥 문자열로 표시하는것으로 하겠다. 기본적으로 기본생성자와 매개변수가 있는 생성자 캡슐화에 의한 setter, getter 는 필드에 맞게 작성해주면 되겠다.

만약 받아온 데이터를 DB 에 저장하려면 테이블도 마찬가지 방법으로 설계하면 되겠다.

```
create table airsatus(
      idx number primary key,
      stationName varchar2(200),
      mangName varchar2(50),
      dataTime date,
      khaiGrade number,
      khaiValue number,
      pm10Grade number,
      pm10Grade1h number,
      pm10Value number,
      pm10Value24 number,
      pm25Grade number,
      pm25Grade1h number,
      pm25Value number,
      pm25Value24 number,
      coGrade number,
      no2Grade number,
      o3Grade number,
      so2Grade number,
      coValue number,
      no2Value number,
      o3Value number,
      so2Value number
);
```

웹브라우저에서 요청 URL 로 파라미터와 함께 보내면 아래와 같이 브라우저에서 반환되는 데이터를 직접확인할 수 있다.

브라우저 URL

http://openapi.airkorea.or.kr/openapi/services/rest/ArpltnInforInqireSvc/getCtprvnRltmMesureDnsty?sidoName=%EC%84%9C%EC%9A%B8&ServiceKey=H1hTYjvwBEqrzWZsXvTi6T8cHV%2FEZ5D3Z7Tr5%2B0AJNeEr4qWZWQ1RB%2FT%2FgIpK0%2FP%2Bw%2BpkoVbJY5oYk%2BpgesyFQ%3D%3D&ver=1.3&_returnType=json

- json 으로 데이터를 받는 방식, service 키는 본인이 받은 키로 입력할 것



1.1 웹서비스 구축하기

공공데이터에 대한 기본적인 분석을 통한 클래스 설계 및 DB설계 등이 완료가 되었으면 서버프로그래밍을 통해서 웹서비스를 구축해보도록 하자. 웹서비스는 두개의 데이터에 대해서 구성하도록하겠다. JSON데이터을 받아 서비스를 구성하는 방법과 XML데이터를 받아서 서비스를 구성하는 방법을 해보겠다.

1.1.2 웹서비스 구성하기

서비스 구성방법

1. **Index.jsp 페이지에서 지역명을 입력하여 버튼으로 대기 정보 요청 요청주소 : "/air.do"**
2. **공공데이터 분석결과 기준으로 AirDto클래스 생성**
3. **air.do주소에 매핑되어 있는 AirController구성**

- 클라이언트 요청을 받으면 지역명을 받아 저장

- HttpURLConnection객체를 이용해서 오픈API에 데이터 요청

- 반환결과인 json파일을 Gson을 이용하여 airDto클래스에 저장

- AirDto를 List로 저장하고 Request에 저장후 result.jsp페이지 반환

4. **result.jsp 구성**

- request데이터를 항목에 맞게 테이블로 구성하여 출력

index.jsp 구성하기

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
```

```
</head>
<body>
	<h2>OpenAPI JSON</h2>
	<form action="<%=request.getContextPath() %>/air.do">
		<input type="text" name="location">
		<input type="hidden" name="param" value="json">
		<input type="submit" value="json요청">
	</form>
</body>
</html>
```

<form>태그를 이용해서 location 값과 json 데이터로 받이 위해 두개의 파라미터를 서버로 보내고 있다.

AirDto.java 구성하기

com.kh.dto.AirDto.java

```
public class AirDto {

	private String stationName;
	private String mangName;
	private String dataTime;
	private int khaiGrade;
	private int khaiValue;
	private int pm10Grade;
	private int pm10Grade1h;
	private int pm10Value;
	private int pm10Value24;
	private int pm25Grade;
	private int pm25Grade1h;
	private int pm25Value;
	private int pm25Value24;
	private int coGrade;
	private int no2Grade;
	private int o3Grade;
	private int so2Grade;
	private double coValue;
	private double no2Value;
	private double o3Value;
	private double so2Value;

	public AirDto() {
	}
	public AirDto(String stationName, String mangName,
```

```
                        String dataTime, int khaiGrade,
                        int khaiValue, int pm10Grade,
                        int pm10Grade1h, int pm10Value,
                        int pm10Value24, int pm25Grade,
                        int pm25Grade1h, int pm25Value,
                        int pm25Value24, int coGrade, int no2Grade,
                        int o3Grade, int so2Grade, double coValue,
                        double no2Value, double o3Value,
                        double so2Value) {
                this.stationName = stationName;
                this.mangName = mangName;
                this.dataTime = dataTime;
                this.khaiGrade = khaiGrade;
                this.khaiValue = khaiValue;
                this.pm10Grade = pm10Grade;
                this.pm10Grade1h = pm10Grade1h;
                this.pm10Value = pm10Value;
                this.pm10Value24 = pm10Value24;
                this.pm25Grade = pm25Grade;
                this.pm25Grade1h = pm25Grade1h;
                this.pm25Value = pm25Value;
                this.pm25Value24 = pm25Value24;
                this.coGrade = coGrade;
                this.no2Grade = no2Grade;
                this.o3Grade = o3Grade;
                this.so2Grade = so2Grade;
                this.coValue = coValue;
                this.no2Value = no2Value;
                this.o3Value = o3Value;
                this.so2Value = so2Value;
        }

        public String getStationName() {
                return stationName;
        }
        public void setStationName(String stationName) {
                this.stationName = stationName;
        }
        public String getMangName() {
                return mangName;
        }
        public void setMangName(String mangName) {
                this.mangName = mangName;
        }
        public String getDataTime() {
                return dataTime;
        }
        public void setDataTime(String dataTime) {
                this.dataTime = dataTime;
```

```
	}
	public int getKhaiGrade() {
		return khaiGrade;
	}
	public void setKhaiGrade(int khaiGrade) {
		this.khaiGrade = khaiGrade;
	}
	public int getKhaiValue() {
		return khaiValue;
	}
	public void setKhaiValue(int khaiValue) {
		this.khaiValue = khaiValue;
	}
	public int getPm10Grade() {
		return pm10Grade;
	}
	public void setPm10Grade(int pm10Grade) {
		this.pm10Grade = pm10Grade;
	}
	public int getPm10Grade1h() {
		return pm10Grade1h;
	}
	public void setPm10Grade1h(int pm10Grade1h) {
		this.pm10Grade1h = pm10Grade1h;
	}
	public int getPm10Value() {
		return pm10Value;
	}
	public void setPm10Value(int pm10Value) {
		this.pm10Value = pm10Value;
	}
	public int getPm10Value24() {
		return pm10Value24;
	}
	public void setPm10Value24(int pm10Value24) {
		this.pm10Value24 = pm10Value24;
	}
	public int getPm25Grade() {
		return pm25Grade;
	}
	public void setPm25Grade(int pm25Grade) {
		this.pm25Grade = pm25Grade;
	}
	public int getPm25Grade1h() {
		return pm25Grade1h;
	}
	public void setPm25Grade1h(int pm25Grade1h) {
		this.pm25Grade1h = pm25Grade1h;
	}
```

```
	public int getPm25Value() {
		return pm25Value;
	}
	public void setPm25Value(int pm25Value) {
		this.pm25Value = pm25Value;
	}
	public int getPm25Value24() {
		return pm25Value24;
	}
	public void setPm25Value24(int pm25Value24) {
		this.pm25Value24 = pm25Value24;
	}
	public int getCoGrade() {
		return coGrade;
	}
	public void setCoGrade(int coGrade) {
		this.coGrade = coGrade;
	}
	public int getNo2Grade() {
		return no2Grade;
	}
	public void setNo2Grade(int no2Grade) {
		this.no2Grade = no2Grade;
	}
	public int getO3Grade() {
		return o3Grade;
	}
	public void setO3Grade(int o3Grade) {
		this.o3Grade = o3Grade;
	}
	public int getSo2Grade() {
		return so2Grade;
	}
	public void setSo2Grade(int so2Grade) {
		this.so2Grade = so2Grade;
	}
	public double getCoValue() {
		return coValue;
	}
	public void setCoValue(double coValue) {
		this.coValue = coValue;
	}
	public double getNo2Value() {
		return no2Value;
	}
	public void setNo2Value(double no2Value) {
		this.no2Value = no2Value;
	}
	public double getO3Value() {
```

```
                return o3Value;
        }
        public void setO3Value(double o3Value) {
                this.o3Value = o3Value;
        }
        public double getSo2Value() {
                return so2Value;
        }
        public void setSo2Value(double so2Value) {
                this.so2Value = so2Value;
        }
}
```

공공데이터에 대한 분석틀 토대로 반환된 데이터를 저장할 수 있는 DTO(VO)를 구성했다.

AirCotroller.java 구성하기 * Sevlet

JSON데이터 파싱을 위한 라이브러리 Gson을 이용하도록하겠다, Gson을 이용하기 위해 프로젝트에 Gson라이브러리를 추가해야 한다.

*Gson추가하기

https://mvnrepository.com/artifact/com.google.code.gson/gson/2.8.6

jar파일 다운로드 후 WEB-INF/lib에 다운로드한 jar파일을 넣어준다.

com.kh.controller.AirController.java

```
package com.kh.controller;

import java.io.IOException;
import java.io.InputStream;
import java.net.HttpURLConnection;
import java.net.MalformedURLException;
import java.net.URL;
import java.net.URLEncoder;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.google.gson.JsonArray;
import com.google.gson.JsonElement;
import com.google.gson.JsonObject;
import com.google.gson.JsonParser;
import com.kh.dto.AirDto;

@WebServlet("/air.do")
public class AirController extends HttpServlet {
	private static final long serialVersionUID = 1L;

	private static final String serviceKey =
	"H1hTYjvwBEqrzWZsXvTi6T8cHV%2FEZ5D3Z7Tr5%2B0AJNeEr4qWZWQ1R
	B%2FT%2FgIpK0%2FP%2Bw%2BpkoVbJY5oYk%2BpgesyFQ%3D%3D";
	//활용신청해서 받은 인증키를 넣으면 된다.
```

```
        protected void doGet(HttpServletRequest request,
HttpServletResponse response)
                        throws ServletException, IOException {

                List<AirDto> list =
request.getParameter("param").equals("json")?openAPIJson(request.
getParameter("location")):"");//XML방식으로 확장하기 위해 작성
                request.setAttribute("list", list);
                //공공데이터에 요청하는 내용은 별도의 openAPIJson()
                //매소드를 구현하여 처리하도록 하겠다.
                RequestDispatcher dispatch =
request.getRequestDispatcher("result.jsp");
                dispatch.forward(request, response);

        }

        protected void doPost(HttpServletRequest request,
HttpServletResponse response)
                        throws ServletException, IOException {
                doGet(request, response);
        }

//공공데이터 요청하는 openAPIJson 매소드
private List<AirDto> openAPIJson(String location) {
                List<AirDto> list = new ArrayList<AirDto>();

                Scanner sc = null;

                try {

                        String url =
"http://openapi.airkorea.or.kr/openapi/services/rest/ArpltnInforI
nqireSvc/getCtprvnRltmMesureDnsty?serviceKey="
                                                + serviceKey
                                                + "&sidoName="
                                                +
URLEncoder.encode(location, "UTF-8")//한글로 들어오는 내용은 UTF-8로
따로 인코딩을 해줘야한다.
                                                + "&_returnType=json";
//_returnType=json을 보내면 json방식으로 데이터를 받을 수 있다.

                        HttpURLConnection urlConnection =
(HttpURLConnection) new URL(url).openConnection();
                        InputStream resp =
urlConnection.getInputStream();
                        sc = new Scanner(resp,"UTF-8");
```

```
                        StringBuffer responseText = new
StringBuffer();
                        while (sc.hasNext()) {
                                responseText.append(sc.next());
                        }

                        JsonElement toJson =
JsonParser.parseString(responseText.toString());
                        JsonArray jsonArray =
toJson.getAsJsonObject().get("list").getAsJsonArray();

                        for (JsonElement item : jsonArray) {
                                JsonObject object =
item.getAsJsonObject();
                                AirDto dto = new AirDto();


        dto.setStationName(object.get("stationName").getAsString()
);

        dto.setMangName(object.get("mangName").getAsString());

        dto.setDataTime(object.get("dataTime").getAsString());

        dto.setKhaiGrade(object.get("khaiGrade").getAsInt());

        dto.setKhaiValue(object.get("khaiValue").getAsInt());

        dto.setPm10Grade(object.get("pm10Grade").getAsInt());

        dto.setPm10Grade1h(object.get("pm10Grade1h").getAsInt());

        dto.setPm10Value(object.get("pm10Value").getAsInt());

        dto.setPm10Value24(object.get("pm10Value24").getAsInt());

        dto.setPm25Grade(object.get("pm25Grade").getAsInt());

        dto.setPm25Grade1h(object.get("pm25Grade1h").getAsInt());

        dto.setPm25Value(object.get("pm25Value").getAsInt());

        dto.setPm25Value24(object.get("pm25Value24").getAsInt());

        dto.setCoGrade(object.get("coGrade").getAsInt());

        dto.setNo2Grade(object.get("no2Grade").getAsInt());
                                try {
```

```
        dto.setO3Grade(object.get("o3Grade").getAsInt());
                                }catch(NumberFormatException e) {
                                        dto.setO3Grade(0);
                                }

        dto.setSo2Grade(object.get("so2Grade").getAsInt());

        dto.setCoValue(object.get("coValue").getAsDouble());

        dto.setNo2Value(object.get("no2Value").getAsDouble());
                                try {

        dto.setO3Value(object.get("o3Value").getAsDouble());
                                }catch(NumberFormatException e) {
                                        dto.setO3Value(0);
                                }

        dto.setSo2Value(object.get("so2Value").getAsDouble());

                                list.add(dto);
                        }

                } catch (MalformedURLException e) {
                        e.printStackTrace();
                } catch (IOException e) {
                        e.printStackTrace();
                } finally {
                        sc.close();
                }

                return list;
        }
}
```

openAPIJson 매소드는 공공데이터를 자바의 HttpURLConnection 객체를 이용해서 요청을 보내고 JSON 형태로 반환된 데이터를 AirDTO.java 객체로 반환해서 List 에 담아 처리하는 기능을 한다.

HttpUrlConnection 객체는 java 에서 web 서버에 HttpRequet 요청을 보내고 그 응답을 스트림으로 받아 올 수 있는 객체이다.

요청을 보낼때는 URL 객체를 생성해서 openConnection()매소드를 이용을한다. URL 객체는 생성지 매개변수로 URL 주소를 작성하면 된다.

openConnection() 매소드가 실행되면 URLConnection 객체가 반환이 되는데 Web 에서 데이터를 받아오기 때문에 최적화되어 있는 HttpURLConnection 객체를 이용해야해서 형변환을 해주고 HttpURLConnection 객체에 대입한다.

HttpURLConnection 객체를 이용해서 응답데이터를 받아오려면 getInputStream()메소드를 이용하면 서버에서 응답한 내용을 String 으로 받아올 수 있다. 스트림을 받아와서 StringBuffer 에 저장한 다음 Gson 으로 파싱하고 AriDto 클래스에 값을 넣어주고 있다.

result.jsp 구성하기 응답페이지 구성하기

jsp 에서는 JSTL core 태그를 사용하기 위해 라이브러리에 해당 jar 파일을 추가해준다. 간단하게 controller 에서 넣어준 데이터를 테이블 형식으로 출력하는 페이지를 구성하겠다.

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>

        <table border="1">
                <tr>
                        <th>측정소명</th>
                        <th>측정망 정보</th>
                        <th>측정일시</th>
                        <th>통합대기환경지수</th>
                        <th>통합대기환경수치</th>
```

```
                <th>미세먼지(PM10) 24시간 등급자료</th>
                <th>미세먼지(  PM10) 1시간 등급자료</th>
                <th>미세먼지(PM10) 농도</th>
                <th>미세먼지(PM10) 24시간 예측이동농도</th>
                <th>미세먼지(PM2.5) 24시간 등급자료</th>
                <th>미세먼지(PM2.5) 1시간 등급자료</th>
                <th>미세먼지(PM2.5) 농도</th>
                <th>미세먼지(PM2.5) 24시간 예측이동농도</th>
                <th>일산화탄소 지수</th>
                <th>이산화질소 지수</th>
                <th>오존 지수</th>
                <th>아황산가스 지수</th>
                <th>일산화탄소 농도</th>
                <th>이산화질소 농도</th>
                <th>오존 농도</th>
                <th>아황산가스 농도</th>
        </tr>
        <c:forEach items="${list }" var="dto">
                <tr>
                        <td>${dto.stationName }</td>
                        <td>${dto.mangName }</td>
                        <td>${dto.dataTime }</td>
                        <td>${dto.khaiGrade }</td>
                        <td>${dto.khaiValue }</td>
                        <td>${dto.pm10Grade }</td>
                        <td>${dto.pm10Grade1h }</td>
                        <td>${dto.pm10Value }</td>
                        <td>${dto.pm10Value24 }</td>
                        <td>${dto.pm25Grade }</td>
                        <td>${dto.pm25Grade1h }</td>
                        <td>${dto.pm25Value }</td>
                        <td>${dto.pm25Value24 }</td>
                        <td>${dto.coGrade }</td>
                        <td>${dto.no2Grade }</td>
                        <td>${dto.o3Grade }</td>
                        <td>${dto.so2Grade }</td>
                        <td>${dto.coValue }</td>
                        <td>${dto.no2Value }</td>
                        <td>${dto.o3Value }</td>
                        <td>${dto.so2Value }</td>
                </tr>
        </c:forEach>
```

```
        </table>

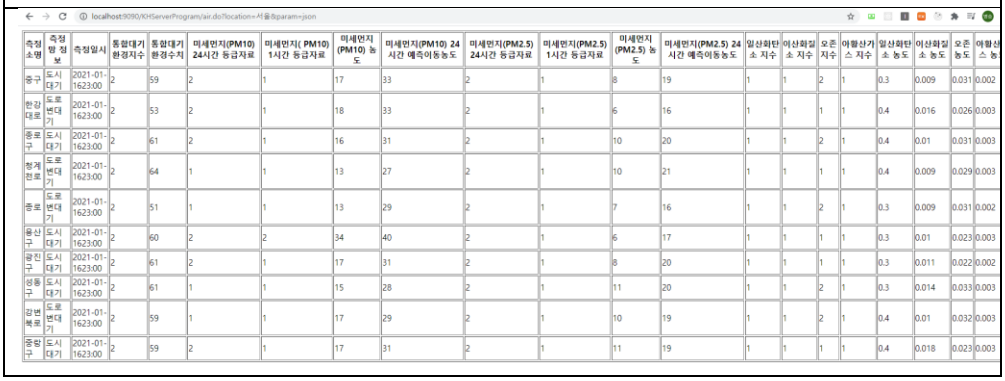
</body>
</html>
```

실행화면

요청화면



결과화면



측정소명	측정망정보	측정일시	통합대기환경지수	통합대기환경수치	미세먼지(PM10) 24시간 등급자료	미세먼지(PM10) 1시간 등급자료	미세먼지(PM10) 농도	미세먼지(PM10) 24시간 예측이동농도	미세먼지(PM2.5) 24시간 등급자료	미세먼지(PM2.5) 1시간 등급자료	미세먼지(PM2.5) 농도	미세먼지(PM2.5) 24시간 예측이동농도	일산화탄소 지수	이산화질소 지수	오존지수	아황산가스 지수	일산화탄소 농도	이산화질소 농도	오존농도	아황산가스 농
중구	도시대기	2021-01-1623:00	2	59	2	1	17	33	2	1	8	19	1	1	2	1	0.3	0.009	0.031	0.002
한강대로	도로변대기	2021-01-1623:00	2	53	2	1	18	33	2	1	6	16	1	1	1	1	0.4	0.016	0.026	0.003
종로구	도시대기	2021-01-1623:00	2	61	2	1	16	31	2	1	10	20	1	1	2	1	0.4	0.01	0.031	0.003
청계천로	도로변대기	2021-01-1623:00	2	64	1	1	13	27	2	1	10	21	1	1	1	1	0.4	0.009	0.029	0.003
종로	도로변대기	2021-01-1623:00	2	51	1	1	13	29	2	1	7	16	1	1	2	1	0.3	0.009	0.031	0.002
용산구	도시대기	2021-01-1623:00	2	60	2	2	34	40	2	1	6	17	1	1	1	1	0.3	0.01	0.023	0.003
광진구	도시대기	2021-01-1623:00	2	61	2	1	17	31	2	1	8	20	1	1	1	1	0.3	0.011	0.022	0.002
성동구	도시대기	2021-01-1623:00	2	61	1	1	15	28	2	1	11	20	1	1	2	1	0.3	0.014	0.033	0.003
강변북로	도로변대기	2021-01-1623:00	2	59	1	1	17	29	2	1	10	19	1	1	2	1	0.4	0.01	0.032	0.003
중랑구	도시대기	2021-01-1623:00	2	59	2	1	17	31	2	1	11	19	1	1	1	1	0.4	0.018	0.023	0.003

공공데이터를 이용하는 방법에 대해 알아봤다. 다른 공공데이터도 유사한 방식으로 URL 요청주소와 parameter 값만 알고있다면 손쉽게 다양한 정보를 가지고 더 다양한 웹애플리케이션을 구현할 수 있다.

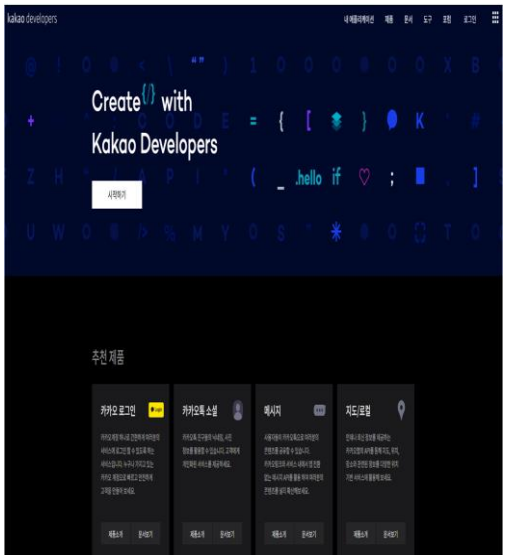
3-2 지오코딩 활용한 지도 API 활용하기

공공데이터와 지오코딩을 적용한 지도 API 연동한 기능구현을 해보자. 공공데이터는 경기데이터 드림에서 제공하는 약국 현황_인허가를 이용하고 지도 API 는 카카오맵을 이용하도록 하자.

1.1 카카오맵 이용하기

카카오맵을 이용하기 위해서는 카카오개발자 사이트에서 회원가입 후 이용하는 서비스를 등록을 해야한다. 카카오개발자 사이트에 접속하자

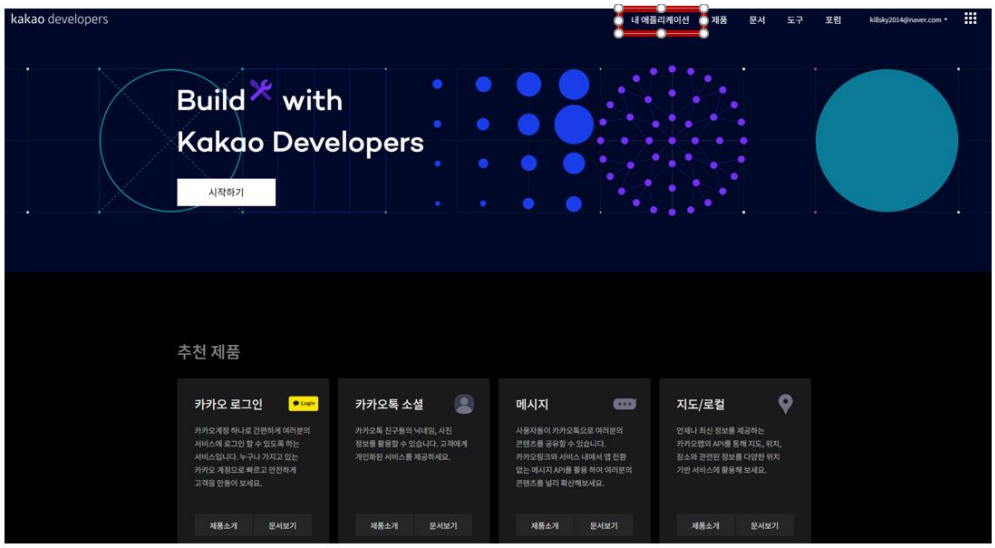
주소 : https:// developers. kakao.com/

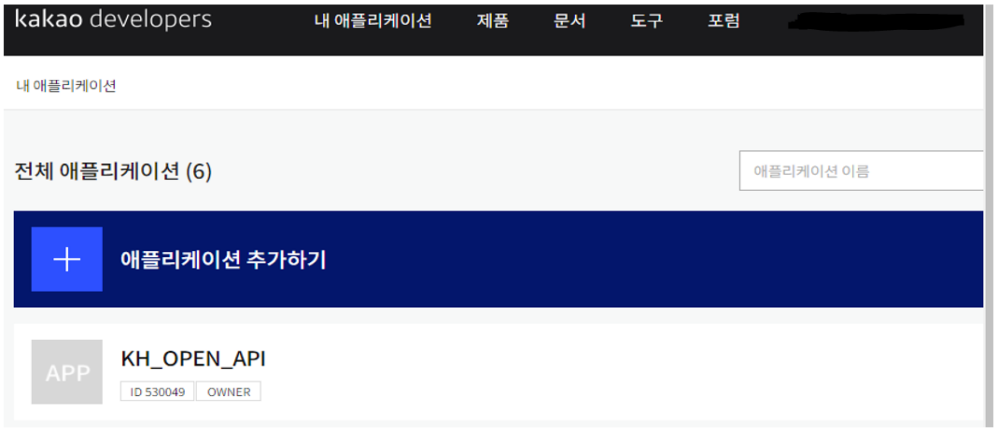






카카오 계정으로 로그인을 할 수 있다. 카카오계정으로 로그인해서 활용해 보자. 로그인을 하고 나면 내 애플리케이션해서 카카오톡이 제공하는 API중 내가 사용할 API를 등록을 해야한다. 등록하는 방법은 아래와 같다.

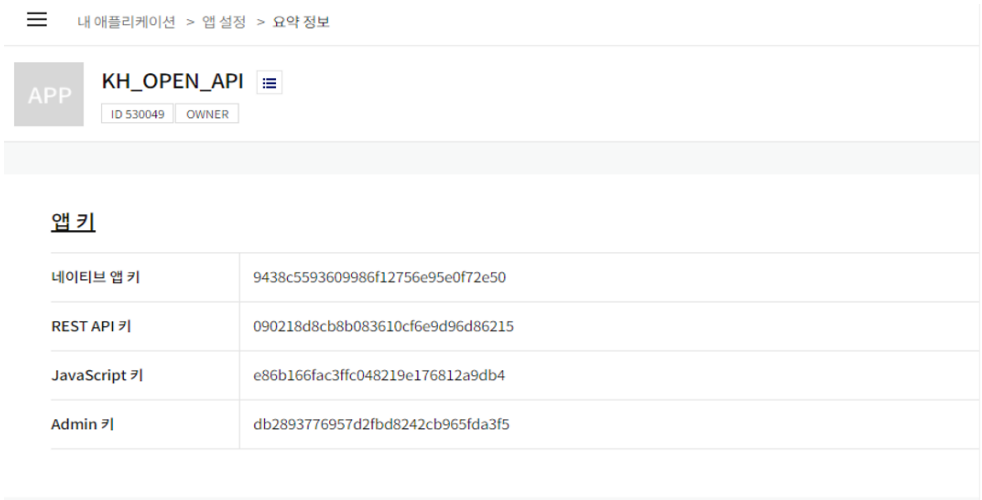




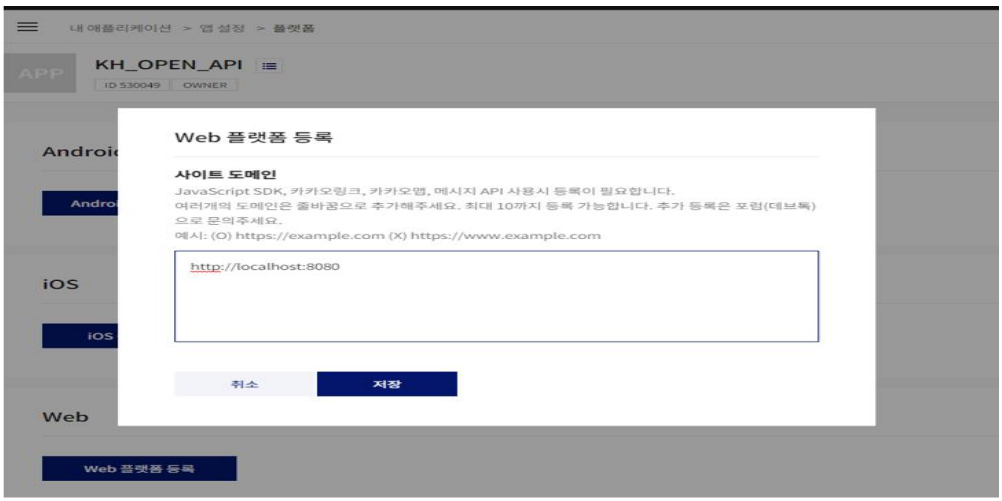


애플리케이션을 추가를 하고 나면 카카오에서 제공하는 API를 사용할 수 있게

인증키를 발급받게 된다. 발급번호는 내 애플리케이션 앱설정 요약정보에서 아래와 같이 확인이 가능하다

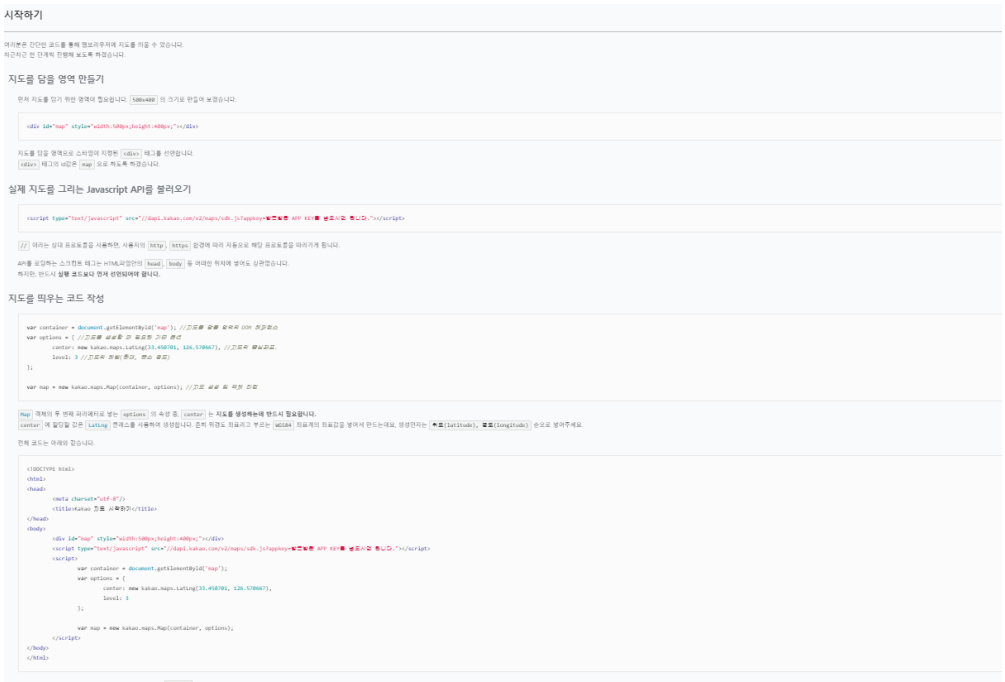


애플리케이션 인증키까지 확인을 했으면 개발할 web의 도메인 주소를 입력해야하는데 톰켓을 사용하여 http://localhost:9090을 등록하면된다. 만약 실서버에 올려 도메인이 있다면 그 도메인 주소도 추가해줘야한다. 사이트 도메인등록이 되어있어야 카카오에서 제공하는 API를 이용할 수 있다.



로그인 후 애플리케이션과 도메인을 등록했하면 javascriptkey를 이용해서 카카오 맵을 이용할 수 있다.

카카오 맵을 이용하는 방법은 카카오맵 사용자 가이드를 이용하면 편리하게 사용할 수가 있다. 주소 https://apis.map.kakao.com/web/ 사이트에서 map관련 API함수를 이용할 수 있다.



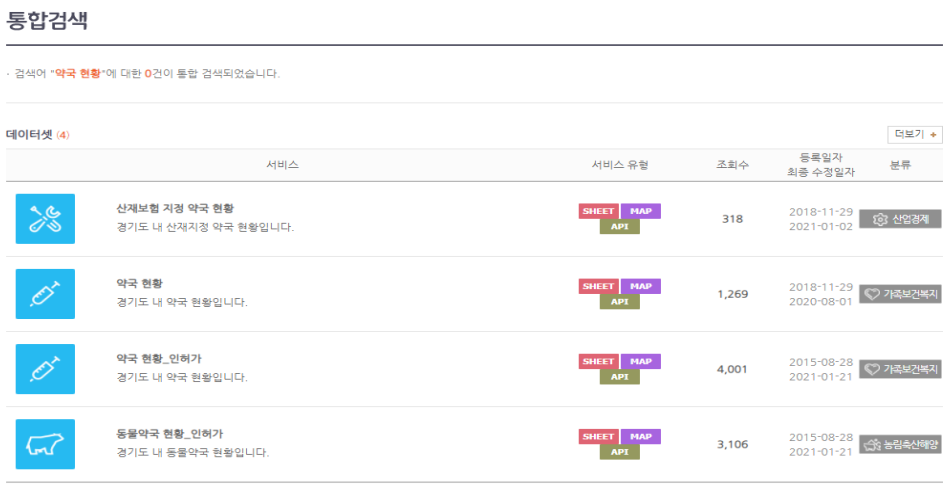
지도를 사용하는 것에 대해서는 실제 애플리케이션으로 만들어 보면서 살펴보자.

1.2 공공데이터 가져오기

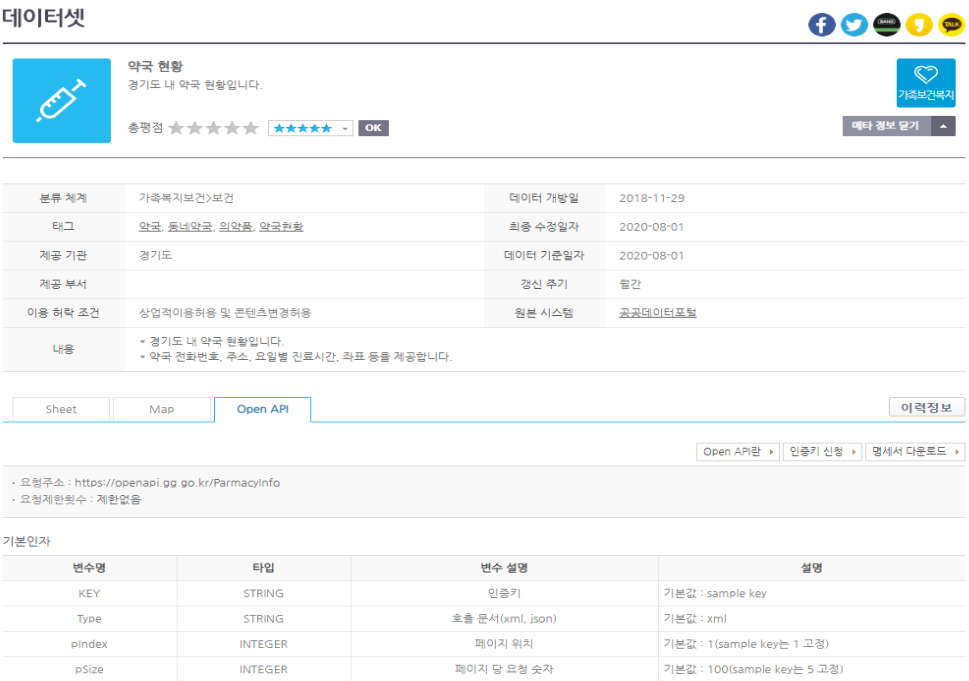
사용할 공공데이터를 검색하여 가져오도록 하자. 공공데이터 중 지도에 표시할 수 있는 데이터로 지정하겠다. 경기데이터 드림에서 제공하는 약국정보를 이용하여 지도 API랑 연동해 보겠다.

경기데이터드림 접속하여(https://data.gg.go.kr/portal/mainPage.do) 약국

현황을 검색하자.



검색결과에서 약국현황으로 들어가서 OpenAPI탭을 선택하고 인증키를 신청한다.



인증키를 신청하여 사용하려면 로그인을 해야하는데 소셜서비스로 가능하

다. 로셜로 로그인하여 활용목적등을 작성하면 신청이 완료되고 바로 인증키는 사용할 수 있다.



사용하기전 데이터명세에서 구조를 분석하고 필요한 데이터를 확인해야 한다.

• 기본인자

변수명	타입	변수 설명	설명
Key	STRING(필수)	인증키	기본값 : sample key
Type	STRING(필수)	호출 문서(xml, json)	기본값 : xml
pIndex	INTEGER(필수)	페이지 위치	기본값 : 1(sample key는 1 고정)
pSize	INTEGER(필수)	페이지 당 요청 숫자	기본값 : 100(sample key는 5 고정)

• 요청인자

변수명	타입	변수 설명	설명
SIGUN_CD	STRING(선택)	시군코드	시군코드
SIGUN_NM	STRING(선택)	시군명	시군명

• 출력값

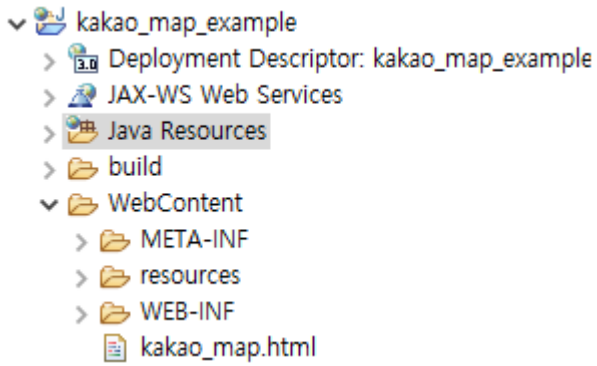
No	출력명	출력 설명	설명
1	SIGUN_CD	시군코드	시군코드
2	SIGUN_NM	시군명	시군명
3	BIZPLC_NM	사업장명	사업장명
4	LICENSG_DE	인허가일자	인허가일자
5	REFINE_ROADNM_ADDR	소재지도로명주소	소재지도로명주소
6	REFINE_LOTNO_ADDR	소재지지번주소	소재지지번주소
7	REFINE_ZIP_CD	소재지우편번호	소재지우편번호
8	REFINE_WGS84_LAT	위도	위도
9	REFINE_WGS84_LOGT	경도	경도
10	LICENSG_CANCL_DE	인허가취소일자	인허가취소일자
11	BSN_STATE_DIV_CD	영업상태구분코드	영업상태구분코드
12	UNITY_BSN_STATE_DIV_CD	통합영업상태구분코드	통합영업상태구분코드
13	UNITY_BSN_STATE_NM	통합영업상태명	통합영업상태명
14	BSN_STATE_NM	영업상태명	영업상태명
15	CLSBIZ_DE	폐업일자	폐업일자
16	SUSPNBIZ_BEGIN_DE	휴업시작일자	휴업시작일자
17	SUSPNBIZ_END_DE	휴업종료일자	휴업종료일자
18	REOPENBIZ_DE	재개업일자	재개업일자
19	LOCPLC_FACLT_TELNO	소재지시설전화번호	소재지시설전화번호
20	LOCPLC_AR_INFO	소재지면적정보	소재지면적정보
21	BIZCOND_DIV_NM_INFO	업태구분명정보	업태구분명정보
22	X_CRDNT_VL	X좌표값	X좌표값
23	Y_CRDNT_VL	Y좌표값	Y좌표값
24	PARMACY_BSN_AR	약국영업면적	약국영업면적
25	APPONT_DE	지정일자	지정일자

제공 데이터 내용 중 지도와 관련 된 위도,경도 값 이외에 지도에 약국관

련 정보를 출력해주기 위해 사업장명, 전화번호, 도로명 주소를 이용하자.

지도API와 공공데이터오픈API를 이용하여 간단한 애플리케이션을 구현해 보자.

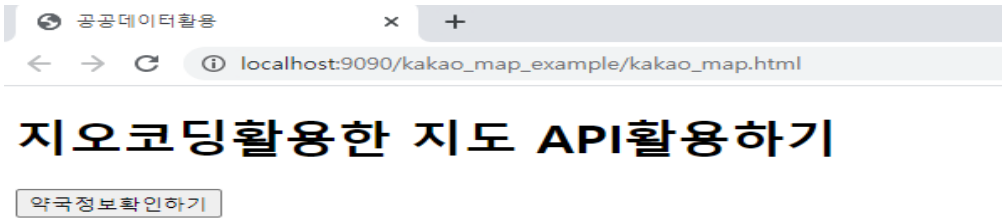
이클립스에서 Dynamic Web 프로젝트를 생성하자.



메인화면 kakao_map.html에서 버튼을 클릭하면 지도 API를 이용해 map을 불러오면서 공공데이터를 불러와서 지도에 공공데이터에 있는 위도, 경도를 이용해서 마커로 지도에 표시하고 관련 정보는 마커정보출력을 이용하겠다. 지도API를 이용해서 위도, 경도로 경로찾기 서비스, 로드뷰를 연결하겠다.

kakao_map.html

```
<!DOCTYPE html>
<html>
<head>
	<meta charset="utf-8">
	<title>다음 지도 API</title>
</head>
<body>
	<h1>지오코딩활용한 지도 API활용하기</h1>
	<button onclick="parmacyInfo();">약국정보확인하기</button>
			<br><br>
<div id="map" style="width:1000px;height:800px;"></div>
</body>
</html>
```



기본 메인화면으로 버튼을 클릭하면 지도가 출력되면서 공공데이터에서 받아온 약국정보를 지도의 마커로 표시하도록 javascript로 구현하였다.

Javascript

```
<script
src="https://dapi.kakao.com/v2/maps/sdk.js?appkey=c3f92e2c59f4752
c157a3d8de9b185dc"></script>
        <script>
        function parmacyInfo(){
                var mapContainer = document.getElementById('map'),
// 지도를 표시할 div
            mapOption = {
                center: new kakao.maps.LatLng(37.57159, 126.97700),
// 지도의 중심좌표
                level: 8, // 지도의 확대 레벨
                mapTypeId : kakao.maps.MapTypeId.ROADMAP // 지도종류
            };
                // 지도를 생성한다
                var map = new kakao.maps.Map(mapContainer,
mapOption);

                //xhr 객체
                let xhr = new XMLHttpRequest();
                //app key
                const API_KEY = '745a539f3967482da7f85d36f3159bf8';
                //parameter 포멧팅
                const buildForm = data => {
                    const form = []
                    for(key in data){
                        form.push(encodeURIComponent(key) + '=' +
encodeURIComponent(data[key]));
                    }
```

```
                return form.join('&')
        }

        //현재 위치 반환

    navigator.geolocation.getCurrentPosition((position)=>{
                var coords = position.coords;
                loadKakaoMap(coords);
        });

        //카카오맵 중심좌표 재설정
        let loadKakaoMap = (coords)=>{
                mapOption.center = new
kakao.maps.LatLng(coords.latitude, coords.longitude)
                getParmacyInfo();
        }

        //약국정보 요청
        let getParmacyInfo = () => {
                let data = new Object;
                data.KEY = API_KEY;
                data.Type = 'json';
                data.pSize = 1000;

                let params = buildForm(data);

    xhr.open('get','https://openapi.gg.go.kr/Parmacy?'+params)
;
                xhr.send();
                xhr.addEventListener('load',parseJsonData);

    xhr.addEventListener('error',()=>{console.dir(xhr.error)})
;
        }


        //xhr응답 바디 파싱
        let parseJsonData = () =>{
                let jsonData = JSON.parse(xhr.response);
                let rowData = jsonData.Parmacy[1].row;
                rowData.forEach((e)=>{
                        if(e.BSN_STATE_NM!='폐업'){
                                console.log(e);
                                let lat = e.REFINE_WGS84_LAT;
                                let lon =
e.REFINE_WGS84_LOGT;
                                //정보구성
```

```
                                        let content="<div
style='padding:5px;'>"+e.BIZPLC_NM;
                                        content+='<br>전화 :
'+e.LOCPLC_FACLT_TELNO;
                                        content+='<br>주소 :
'+e.REFINE_ROADNM_ADDR;
                                        content+="<br><a
href='https://map.kakao.com/link/to/"+e.BIZPLC_NM+","+lat+","+lon
+"' target='_blank'>길찾기연결</a>";
                                        content+="  <a
href='https://map.kakao.com/link/roadview/"+lat+","+lon+"'
target='_blank'>로드뷰</a>";
                                        content+="</div>";
                                        loadMarker(lat,lon,content);
                                }
                        })
                }

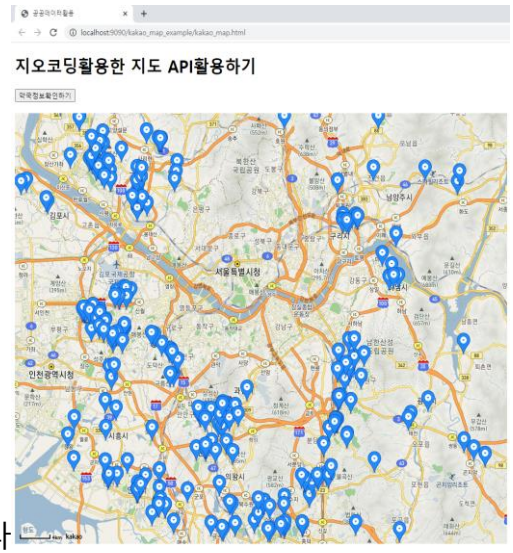
                //마커출력
                let loadMarker = (lat,lon, content) =>{
                        // 지도에 마커를 생성하고 표표시한다
                        var marker = new kakao.maps.Marker({
                            position: new kakao.maps.LatLng(lat,
lon), // 마커의 좌표

                            map: map, // 마커를 표시할 지도 객체
                            clickable:true
                        });
                        var infowindow=new kakao.maps.InfoWindow({
                                content:content,
                                removable:true
                        })
                        //마커 클릭이벤트주기

        kakao.maps.event.addListener(marker,'click',function(){
                                infowindow.open(map,marker);
                        })
                }
        }
        </script>
```

먼저 지도API를 사용하기 위해 <script src="*https://dapi.kakao.com/v2/maps/sdk.js?appkey=c3f92e2c59f4752c157a3d8de9b185dc*"></script> 로 제공하는 js파일을 받아온다. js파일은 인증키가 필요하고

파라미터 appkey에 카카오 디벨로퍼에서 받아온 키를 입력하면 된다.
<button>을 클릭하면 paramacyInfo()함수가 실행될 수 있게 이벤트를 연결하고 paramacyInfo()함수는 지도API를 이용해서 지도를 불러오면서 공공데이터의 오픈API를 이용해서 데이터를 받아오는 기능을 한다.
약국현황에 대한 공공데이터는 JSON으로 데이터가 전송되고있어 필요한 데이터를 parseJsonData함수를 이용해서 파싱하여 사용하고 있다.
파싱된 데이터는 loadMarker함수를 이용해서 지도에 마커로 표시하게 된다.



.



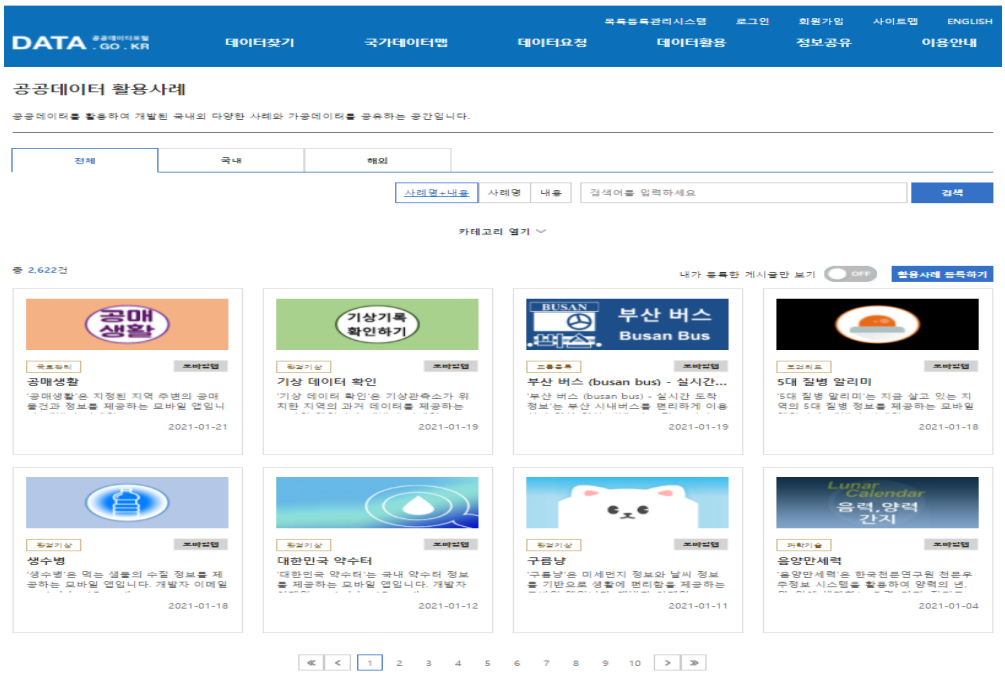
공공데이터로 가져온 약국의 위치가 지도에 표시가 되고 클릭하면 약국에 대한 정보가 출력되고 길찾기연결, 로드뷰를 클릭하면 해당 서비스로 연결되게 구성해보았다. 이렇게 지오코딩과 공공데이터를 융합하면 더 나은 웹애플리케이션 서비스를 제공할 수가 있다

3-3 공공데이터 활용사례 연구를 통한 웹애플리케이션 적용하기

공공데이터 포털에서 제공하는 공공데이터 활용사례를 확인하고 관련내용을 웹애플키케이션에 적용해보자.

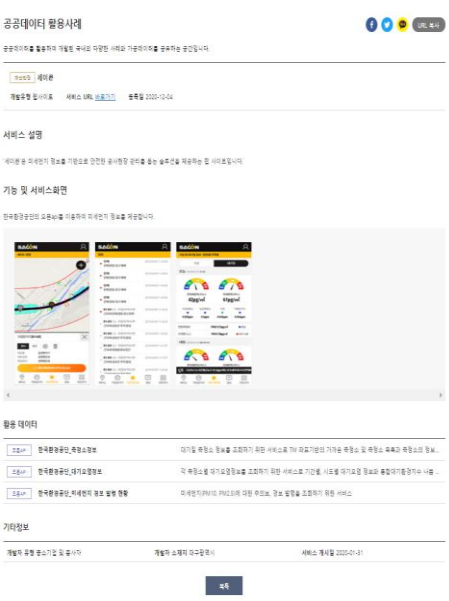
1.1 공공데이터 활용사례

공공데이터 활용사례는 공공데이터 포털에서 제공해주고있다.

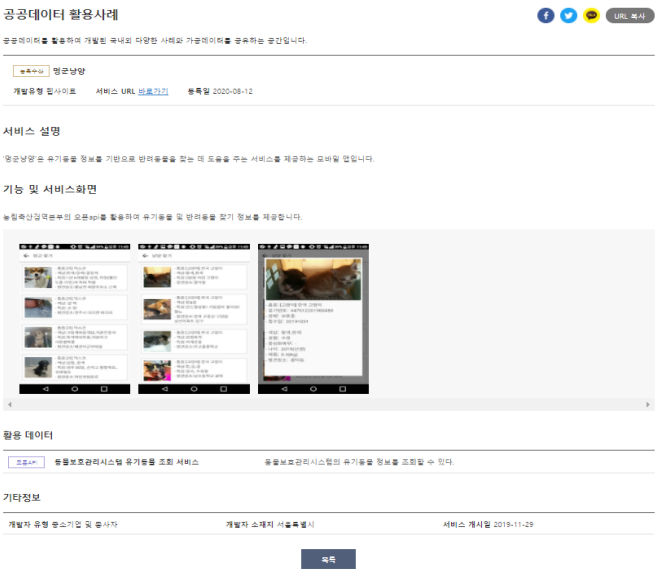


공공데이터 포털에서 제공하는 공공데이터 활용사례를 확인하고 관련내용을 웹애플키케이션에 적용해보자 활용사례를 보면 모바일앱, 웹사이트에서 이용한 것을 확인할 수 있다. 그중 웹애플케이션에 적용하기 위해 웹애프리케이션에 적용된 내용을 살펴보자.





미세먼지정보를 기반으로 마스크자판기 운영을 돕는 서비스를 제공하는 사이트와 공사현장관리를 돕는 솔루션을 제작하였고, 두곳 모두 사용데이터는 한국환경공단_측정소정보, 한국환경공단_대기오염정보, 한국환경공단_미세먼지 경보발령현황을 이용하였다.



유기동물정보를 기반으로 반려동물을 찾는데 도움을 주는 서비스를 제공하는 것

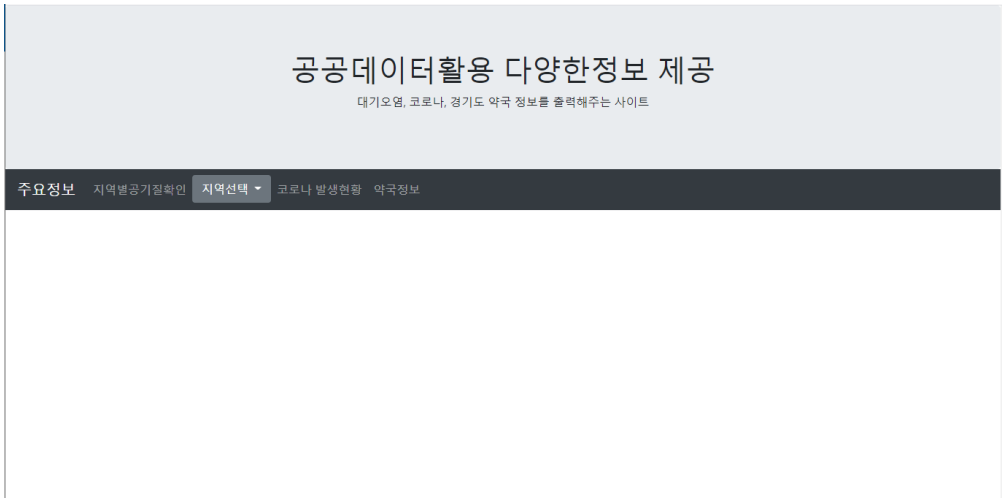
으로 동물보호관리시스템 유기동물 조회서비스를 사용하였다.

1.2 공공데이터기반 웹애플리케이션 구현하기

다양한 공공데이터를 이용해서 종합정보를 제공하는 애플리케이션을 구현해보자. 공공데이터 중 시도별 실시간 대기환경정보, 경기 데이터 드림에서 제공하는 약국현황데이터, 코로나 감염자현황데이터를 제공하는 웹애플리케이션을 만들어 보자.

1.2.1 메인화면구성

메인화면은 지역별 공기질확인, 코로나 발생현황, 약국정보 총 세개의 메뉴로 구성하고 구성화면은 다음과 같다. 각 메뉴를 click이벤트를 주어 공공데이터와 연결되어 각각의 정보를 출력하게 하고, 약국정보는 카카오지도 API를 이용하여 지도에 표시하는 것으로 하자.



Index.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
  <title>대기오염상황사이트</title>
  <meta charset="utf-8">
```

```
  <meta name="viewport" content="width=device-width, initial-
scale=1">
  <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/css/bootstr
ap.min.css">
  <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.mi
n.js"></script>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.16.0/umd/
popper.min.js"></script>
  <script
src="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap
.min.js"></script>
  <style>
  .fakeimg {
    height: 200px;
    background: #aaa;
  }
  </style>
</head>
<body>

<div class="jumbotron text-center" style="margin-bottom:0">
  <h1>공공데이터활용 다양한정보 제공</h1>
  <p>대기오염, 코로나, 경기도 약국 정보를 출력해주는 사이트</p>
</div>

<nav class="navbar navbar-expand-sm bg-dark navbar-dark">
  <a class="navbar-brand" href="#">주요정보</a>
  <button class="navbar-toggler" type="button" data-
toggle="collapse" data-target="#collapsibleNavbar">
    <span class="navbar-toggler-icon"></span>
  </button>
  <div class="collapse navbar-collapse" id="collapsibleNavbar">
    <ul class="navbar-nav">
      <li class="nav-item">
        <a class="nav-link"
href="javascript:void(0)"">지역별공기질확인</a>
      </li>
      <div class="dropdown">
                <button type="button" class="btn btn-secondary
dropdown-toggle" data-toggle="dropdown">
                  지역선택
                </button>
                <div class="dropdown-menu">
                  <a class="dropdown-item"
```

```
href="javascript:void(0)" onclick="airConnect(event);">서울</a>
                    <a class="dropdown-item"
href="javascript:void(0)" onclick="airConnect(event);">부산</a>
                    <a class="dropdown-item"
href="javascript:void(0)" onclick="airConnect(event);">대구</a>
                    <a class="dropdown-item"
href="javascript:void(0)" onclick="airConnect(event);">인천</a>
                    <a class="dropdown-item"
href="javascript:void(0)" onclick="airConnect(event);">광주</a>
                    <a class="dropdown-item"
href="javascript:void(0)" onclick="airConnect(event);">대전</a>
                    <a class="dropdown-item"
href="javascript:void(0)" onclick="airConnect(event);">경기</a>
                    <a class="dropdown-item"
href="javascript:void(0)" onclick="airConnect(event);">강원</a>
                    <a class="dropdown-item"
href="javascript:void(0)" onclick="airConnect(event);">충북</a>
                    <a class="dropdown-item"
href="javascript:void(0)" onclick="airConnect(event);">충남</a>
                    <a class="dropdown-item"
href="javascript:void(0)" onclick="airConnect(event);">전북</a>
                    <a class="dropdown-item"
href="javascript:void(0)" onclick="airConnect(event);">전남</a>
                    <a class="dropdown-item"
href="javascript:void(0)" onclick="airConnect(event);">경남</a>
                    <a class="dropdown-item"
href="javascript:void(0)" onclick="airConnect(event);">제주</a>
                    <a class="dropdown-item"
href="javascript:void(0)" onclick="airConnect(event);">경북</a>
                    <a class="dropdown-item"
href="javascript:void(0)" onclick="airConnect(event);">세종</a>
                  </div>
            </div>
      <li class="nav-item">
        <a class="nav-link" href="javascript:void(0)"
onclick="corona19();">코로나 발생현황</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" href="javascript:void(0)"
onclick="parmacyInfo();">약국정보</a>
      </li>
```

```
    </ul>
  </div>
</nav>
<section id="container" style="width:100%;height:700px;">
</section>
</body>
</html>
```

디자인은 부트스트랩을 적용하여 스타일을 디자인하자. 부트스트랩은 css와 script를 외부에서 불러와서 적용하고 각 클래스별 디자인과 이벤트가 연결되어 있어 사용하기가 편리하다. 메뉴와 해더부분에 작성했다. 각 메뉴 클릭했을 때 데이터는 가져오는 방식은 ajax를 이용하여 적용하자 페이지 변경이 없어 깔끔하다.

1.2.2 지역별 공기질 확인

지역별 공기질확인 script

```
function airConnect(e){
		console.log(e.target);

	$.get("${pageContext.request.contextPath}/air.do?location="+e.target.innerHTML,function(data){
			setAirTable(JSON.parse(data));

		});
	}
	function setAirTable(data){
		$("#container").html("");
		console.log(data);
		let table=$("<table>");
		let head=$("<tr>");

	head.append("<th>측정소명</th>").append("<th>통합대기환경지수</th>").append("<th>통합대기환경수치</th>").append("<th>미세먼지(PM10)등급</th>")
		.append("<th>미세먼지(PM2.5)등급</th>");
		table.append(head);
		for(let i=0;i<data.length;i++){
			let tr=$("<tr>");

	tr.append($("<td>").html(decodeURI(data[i].stationName))).append($("<td>").html(data[i].khaiGrade)).append($("<td>").html(data[i].khaiValue))
			.append($("<td>").html(data[i].pm10Grade))
```

```
                        .append($("<td>").html(data[i].pm25Grade));
                        table.append(tr);
                }
                $("#container").html(table);
        }
```

Ajax로 요청을 보내서 비동기식으로 처리하고 요청은 서버로 보내야한다. 직접 요청주소로 보낼경우 cors에러가 발생하여 was로 보내서 처리하도록하게 했다. 요청시 지역에 따라 결과를 받아오기 위해 요청 파라미터로 지역을 보냈다. 서버에서 응답은 JSON방식으로 전달되는데 전달하는 과정에서 한글이 깨지는 증상이 발생할 수 있다. 그렇기 때문에 서버에서 URLEncode.encode()매소드를 이용해서 인코딩한 값을 프론트에서 decodeURI()함수로 다시 디코딩하여 한글깨짐을 방지해야 한다. JSON데이터로 가져온 값을 화면의 테이블로 만들어주고 만들어진 테이블은 <section>태그에 출력해줬다.

지역별 공기질확인 서블릿 AirController.java

```java
package com.kh.controller;

import java.io.IOException;
import java.io.InputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.net.URLEncoder;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.google.gson.Gson;
import com.google.gson.JsonArray;
import com.google.gson.JsonElement;
import com.google.gson.JsonObject;
import com.google.gson.JsonParser;
import com.kh.dto.AirDto;

@WebServlet("/air.do")
public class AirController extends HttpServlet {
        private static final long serialVersionUID = 1L;
```

```
	private static final String serviceKey =
"H1hTYjvwBEqrzWZsXvTi6T8cHV%2FEZ5D3Z7Tr5%2B0AJNeEr4qWZWQ1RB%2FT%2
FgIpK0%2FP%2Bw%2BpkoVbJY5oYk%2BpgesyFQ%3D%3D";

	protected void doGet(HttpServletRequest request,
HttpServletResponse response)
			throws ServletException, IOException {
		Scanner sc=null;
		String url =
"http://openapi.airkorea.or.kr/openapi/services/rest/ArpltnInforI
nqireSvc/getCtprvnRltmMesureDnsty?serviceKey="
				+ serviceKey
				+ "&sidoName="
				+
URLEncoder.encode(request.getParameter("location"), "UTF-8")
				+"&numOfRows=100"
				+ "&_returnType=json";

		HttpURLConnection urlConnection =
(HttpURLConnection) new URL(url).openConnection();
		InputStream resp = urlConnection.getInputStream();
		sc = new Scanner(resp);
		List<AirDto> list=new ArrayList();
		StringBuffer responseText = new StringBuffer();
		while (sc.hasNext()) {
			responseText.append(sc.next());
		}

		JsonElement toJson =
JsonParser.parseString(responseText.toString());
		JsonArray jsonArray =
toJson.getAsJsonObject().get("list").getAsJsonArray();

		for (JsonElement item : jsonArray) {
			JsonObject object = item.getAsJsonObject();
			AirDto dto = new AirDto();


	dto.setStationName(URLEncoder.encode(object.get("stationNa
me").getAsString(),"UTF-8"));

	dto.setMangName(URLEncoder.encode(object.get("mangName").g
etAsString(),"UTF-8"));

	dto.setDataTime(object.get("dataTime").getAsString());

	dto.setKhaiGrade(object.get("khaiGrade").getAsString());

	dto.setKhaiValue(object.get("khaiValue").getAsString());
```

```
        dto.setPm10Grade(object.get("pm10Grade").getAsString());

        dto.setPm10Grade1h(object.get("pm10Grade1h").getAsString()
);

        dto.setPm10Value(object.get("pm10Value").getAsString());

        dto.setPm10Value24(object.get("pm10Value24").getAsString()
);

        dto.setPm25Grade(object.get("pm25Grade").getAsString());

        dto.setPm25Grade1h(object.get("pm25Grade1h").getAsString()
);

        dto.setPm25Value(object.get("pm25Value").getAsString());

        dto.setPm25Value24(object.get("pm25Value24").getAsString()
);

        dto.setCoGrade(object.get("coGrade").getAsString());

        dto.setNo2Grade(object.get("no2Grade").getAsString());

        dto.setO3Grade(object.get("o3Grade").getAsString());

        dto.setSo2Grade(object.get("so2Grade").getAsString());

        dto.setCoValue(object.get("coValue").getAsString());

        dto.setNo2Value(object.get("no2Value").getAsString());

        dto.setO3Value(object.get("o3Value").getAsString());

        dto.setSo2Value(object.get("so2Value").getAsString());

                        list.add(dto);
                }
                sc.close();
                new Gson().toJson(list,response.getWriter());
        }

        protected void doPost(HttpServletRequest request,
HttpServletResponse response)
                        throws ServletException, IOException {
                doGet(request, response);
        }
}
```

Client가 ajax로 요청한 내용을 응답해주는 서블릿으로 공공데이터 서버에서 데이터를 가져와 데이터를 파싱처리하여 JSON방식으로 보내주고 있다 있때 앞서 말한것처럼 한글문자가 깨지는 증상이 있어 URIEncode.encode()매소드를 이용해서 인코딩하고 있고, JSON 라이브러리인 GSON을 이용하여 한번이 List데이터를 보내고 있다. 객체의 자료형은 모두 스트링형으로 하자. 실시간으로 변경되는 데이터로 값이 안들어오거나 int, double형으로 변경이 안되 NumberFormatException이 발생하기 때문이다. 구현하고 실행하면 정상적으로 지역별로 데이터가 테이블 형식으로 출력되는 것을 볼 수 있다.



대기오염수치에 따라 라인색을 변경해주는 디자인을 해도 좋다.

1.2.3 코로나 감염현황

코로나 감연현황 script

```
function corona19(){

	$.get("${pageContext.request.contextPath}/corona19",functi
on(data){
			//console.log(data);
			setTable(JSON.parse(data));
		})
	}
	function setTable(data){
```

```
                $("#container").html("");
                let table=$("<table class='table table-striped'>");
                table.append($("<thead>"));
                let head=$("<tr>");

        head.append("<th>시도명</th>").append("<th>확진자수</th>").
append("<th>지역감염</th>").append("<th>해외유입</th>")
                .append("<th>격리중
환자수</th>").append("<th>전월대비증감수</th>");
                table.append(head);
                table.append($("<tbody>"));
                for(let i=0;i<data.length;i++){
                        let tr=$("<tr>");
                        console.log(decodeURI(data[i].gubun));

        tr.append($("<td>").html(decodeURI(data[i].gubun))).append
($("<td>").html(data[i].defCnt)).append($("<td>").html(data[i].lo
calOccCnt))
                        .append($("<td>").html(data[i].overFlowCnt))

        .append($("<td>").html(data[i].isolIngCnt)).append($("<td>
").html(data[i].incDec));
                        table.append(tr);
                }
                $("#container").html(table);
        }
```

지역별 공기질확인과 유사하게 ajax를 이용해서 서버로 요청을 하였고 별로 요청 데이터는 없다. 코로나데이터도 마찬가지로 JSON으로 데이터를 받게 되는데 한 한글인 경우 깨져서 서버에서 URLEncoder.encode()매소드를 이용해서 인코딩하고 프론트단에서 decode하여 한글이 깨지지않게 설정했고 JSON으로 가져온 데이터를 파싱하여 setTable함수를 이용해서 테이블로 데이터를 출력하고 있다.

코로나데이터확인 서블릿 CoronaStatus.java

```
package com.kh.controller;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.UnsupportedEncodingException;
import java.net.HttpURLConnection;
import java.net.MalformedURLException;
import java.net.URL;
import java.net.URLEncoder;
```

```
import java.util.ArrayList;
import java.util.Calendar;
import java.util.Date;
import java.util.GregorianCalendar;
import java.util.List;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;

import org.w3c.dom.Document;
import org.w3c.dom.Node;
import org.w3c.dom.NodeList;
import org.xml.sax.SAXException;

import com.google.gson.Gson;
import com.kh.dto.Corona19Data;

/**
 * Servlet implementation class CoronaStatus
 */
@WebServlet("/corona19")
public class CoronaStatus extends HttpServlet {
	private static final long serialVersionUID = 1L;

    /**
     * @see HttpServlet#HttpServlet()
     */
    public CoronaStatus() {
        super();
        // TODO Auto-generated constructor stub
    }

	/**
	 * @see HttpServlet#doGet(HttpServletRequest request,
HttpServletResponse response)
	 */
	protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
		// TODO Auto-generated method stub
		GregorianCalendar today=new GregorianCalendar();
		String day="";
		day+=today.get(Calendar.YEAR);
```

```
		day+=String.valueOf(today.get(Calendar.MONTH)+1).length()!
=2?"0"+String.valueOf(today.get(Calendar.MONTH)+1):String.valueOf
(today.get(Calendar.MONTH)+1);
			day+=today.get(Calendar.DATE)-1;
			System.out.println(day);
			//1. 요청 url 생성
		StringBuilder urlBuilder = new
StringBuilder("http://openapi.data.go.kr/openapi/service/rest/Cov
id19/getCovid19SidoInfStateJson"); /*URL*/
		try {
			urlBuilder.append("?" +
URLEncoder.encode("ServiceKey","UTF-8") +
"=xG7FbVRiB%2FVRULA5s7YlsBsZOsoP1a%2BS%2BdHFBLAaDX%2FRj3QgOTytvSM
vI7EbGXFYK0yIDMpKM97N9%2F9awD9dYg%3D%3D");/*Service Key*/
			urlBuilder.append("&" +
URLEncoder.encode("pageNo","UTF-8") + "=" +
URLEncoder.encode("1", "UTF-8")); /*페이지번호*/
			urlBuilder.append("&" +
URLEncoder.encode("numOfRows","UTF-8") + "=" +
URLEncoder.encode("10", "UTF-8")); /*한 페이지 결과 수*/
			urlBuilder.append("&" +
URLEncoder.encode("startCreateDt","UTF-8") + "=" +
URLEncoder.encode(day, "UTF-8")); /*검색할 생성일 범위의 시작*/
			urlBuilder.append("&" +
URLEncoder.encode("endCreateDt","UTF-8") + "=" +
URLEncoder.encode(day, "UTF-8")); /*검색할 생성일 범위의 종료*/
		} catch (UnsupportedEncodingException e) {
			e.printStackTrace();
		}

		//2. connection객체 생성
		URL url = null;
		HttpURLConnection conn = null;
			try {
			url = new URL(urlBuilder.toString());
			conn = (HttpURLConnection)
url.openConnection();
			conn.setRequestMethod("GET");
			conn.setRequestProperty("Content-type",
"application/json");
			System.out.println("Response code: " +
conn.getResponseCode());
		} catch (MalformedURLException e) {
			e.printStackTrace();
		} catch (IOException e) {
			// TODO Auto-generated catch block
```

```
                        e.printStackTrace();
                }
            List<Corona19Data> list = new ArrayList<>();
                //3. 요청전송 및 응답 처리
        BufferedReader br = null;
                try {
                        int statusCode = conn.getResponseCode();
                        if(statusCode >= 200 &&  statusCode <= 300)
                                br = new BufferedReader(new
InputStreamReader(conn.getInputStream()));
                        else
                                br = new BufferedReader(new
InputStreamReader(conn.getErrorStream()));

                        Document doc =
parseXML(conn.getInputStream());

                        //a. item 태그객체 목록으로 가져온다.
                        NodeList descNodes =
doc.getElementsByTagName("item");

                        //b. Corona19Data List객체 생성
                    //List<Corona19Data> list = new ArrayList<>();

                        //c. 각 item태그의 자식태그에서 정보 가져오기
                    for(int i = 0; i < descNodes.getLength(); i++){
                                        //item
                        Node item = descNodes.item(i);
                        Corona19Data data = new Corona19Data();

                        //item 자식태그에 순차적으로 접근
                        for(Node node = item.getFirstChild();
node != null; node = node.getNextSibling()){ //첫번째 자식을
시작으로 마지막까지 다음 형제를 실행

        //System.out.println(node.getNodeName() + " : " +
node.getTextContent());
                                switch(node.getNodeName()) {
                                case "seq":
data.setSeq(Integer.parseInt(node.getTextContent())); break;
                                case "createDt":
data.setCreateDt(node.getTextContent()); break;
                                case "deatCnt":
data.setDeathCnt(Integer.parseInt(node.getTextContent())); break;
                                case "defCnt":
data.setDefCnt(Integer.parseInt(node.getTextContent())); break;
```

```
                                case "gubun":
data.setGubun(URLEncoder.encode(node.getTextContent(),"UTF-8"));
break;
                                case "gubunCn":
data.setGubunCn(URLEncoder.encode(node.getTextContent(),"UTF-
8")); break;
                                case "gubunEn":
data.setGubunEn(URLEncoder.encode(node.getTextContent(),"UTF-
8")); break;
                                case "incDec":
data.setIncDec(Integer.parseInt(node.getTextContent())); break;
                                case "isolClearCnt":
data.setIsolClearCnt(Integer.parseInt(node.getTextContent()));
break;
                                case "isolIngCnt":
data.setIsolIngCnt(Integer.parseInt(node.getTextContent()));
break;
                                case "localOccCnt":
data.setLocalOccCnt(Integer.parseInt(node.getTextContent()));
break;
                                case "overFlowCnt":
data.setOverFlowCnt(Integer.parseInt(node.getTextContent()));
break;
                                case "qurRate":
data.setQurRate(node.getTextContent()); break;
                                case "stdDay":
data.setStdDay(node.getTextContent()); break;
                                case "updateDt":
data.setUpdateDt(node.getTextContent()); break;
                                }
                        }

                                //d. List객체에 추가
                        list.add(data);

                    }
                } catch (IOException e) {
                        e.printStackTrace();
                } finally {
                        try {
                                br.close();
                        } catch (Exception e) {
                                e.printStackTrace();
                        }
                }
                new Gson().toJson(list, response.getWriter());
        }

        /**
```

```
         * @see HttpServlet#doPost(HttpServletRequest request,
HttpServletResponse response)
         */
        protected void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {
                // TODO Auto-generated method stub
                doGet(request, response);
        }

        private Document parseXML(InputStream stream) {

        DocumentBuilderFactory objDocumentBuilderFactory =
DocumentBuilderFactory.newInstance();
        DocumentBuilder objDocumentBuilder = null;
        Document doc = null;

        try {
                        objDocumentBuilder =
objDocumentBuilderFactory.newDocumentBuilder();
                        doc = objDocumentBuilder.parse(stream);
                } catch (ParserConfigurationException e) {
                                e.printStackTrace();
                } catch (SAXException e) { //Simple API for XML
                        e.printStackTrace();
                } catch (IOException e) {
                        e.printStackTrace();
                }

        return doc;
    }
}
```

코로나 데이터는 XML형태로 받게 되어 xml파싱메소드를 이용해서 데이터를 파싱한다 이때 자바에서 제공하는 Document객체를 이용한다. JSON으로 전송하기 위해 GSON을 사용하였고 한글이 깨지지 않게 하기 위해 URLEncoder.encode()매소드를 사용하였다. 그리고 공공데이터 요청시에 정보에 대한 날짜를 요청데이터로 보내야하는데 하루전 날짜를 보낼 수 있도록 하였다.



공공데이터활용 다양한정보 제공

대기오염, 코로나, 경기도 약국 정보를 출력해주는 사이트

주요정보 지역별공기질확인 지역선택 코로나 발생현황 약국정보

시도명	확진자수	지역감염	해외유입	격리중 환자수	전일대비증감수
검역	2633	0	9	736	9
제주	513	2	0	29	2
경남	1854	21	0	286	21
경북	2833	12	3	214	15
전남	699	7	0	124	7
전북	1002	0	0	97	0
충남	1943	7	0	163	7
충북	1489	1	0	254	1
강원	1590	8	1	242	9
경기	18771	102	14	3320	116
세종	173	0	0	16	0
울산	909	2	1	92	3
대전	959	1	0	48	1
광주	1471	2	0	180	2

공공데이터에서 제공하는 파일중 필요한 파일만 필터해서 가져와 출력해주고 있다.

1.2.2 약국정보 출력하기

약국정보는 카카오 지도 API와 연결하여 지도에 약국에 위치를 표시하고 정보를 출력하자.

약국정보 요청 스트립트

```
        function parmacyInfo(){
                var mapContainer =
document.getElementById('container'), // 지도를 표시할 div
            mapOption = {
                center: new kakao.maps.LatLng(37.57159, 126.97700),
// 지도의 중심좌표
                level: 8, // 지도의 확대 레벨
                mapTypeId : kakao.maps.MapTypeId.ROADMAP // 지도종류
            };
                // 지도를 생성한다
                var map = new kakao.maps.Map(mapContainer,
mapOption);

                //xhr 객체
                let xhr = new XMLHttpRequest();
                //app key
                const API_KEY = '745a539f3967482da7f85d36f3159bf8';
```

```
                //parameter 포멧팅
                const buildForm = data => {
                    const form = []
                    for(key in data){
                         form.push(encodeURIComponent(key) + '=' +
encodeURIComponent(data[key]));
                    }

                    return form.join('&')
                }

                //현재 위치 반환

        navigator.geolocation.getCurrentPosition((position)=>{
                        var coords = position.coords;
                        loadKakaoMap(coords);
                });

                //카카오맵 중심좌표 재설정
                let loadKakaoMap = (coords)=>{
                        mapOption.center = new
kakao.maps.LatLng(coords.latitude, coords.longitude)
                        getParmacyInfo();
                }

                //약국정보 요청
                let getParmacyInfo = () => {
                        let data = new Object;
                        data.KEY = API_KEY;
                        data.Type = 'json';
                        data.pSize = 1000;

                        let params = buildForm(data);

        xhr.open('get','https://openapi.gg.go.kr/Parmacy?'+params)
;
                        xhr.send();
                        xhr.addEventListener('load',parseJsonData);

        xhr.addEventListener('error',()=>{console.dir(xhr.error)})
;
                }


                //xhr응답 바디 파싱
                let parseJsonData = () =>{
                        let jsonData = JSON.parse(xhr.response);
```

```
                        let rowData = jsonData.Parmacy[1].row;
                        rowData.forEach((e)=>{
                                if(e.BSN_STATE_NM!='폐업'){
                                        console.log(e);
                                        let lat = e.REFINE_WGS84_LAT;
                                        let lon =
e.REFINE_WGS84_LOGT;
                                        //정보구성
                                        let content="<div
style='padding:5px;'>"+e.BIZPLC_NM;
                                        content+='<br>전화 :
'+e.LOCPLC_FACLT_TELNO;
                                        content+='<br>주소 :
'+e.REFINE_ROADNM_ADDR;
                                        content+="<br><a
href='https://map.kakao.com/link/to/"+e.BIZPLC_NM+","+lat+","+lon
+"' target='_blank'>길찾기연결</a>";
                                        content+="  <a
href='https://map.kakao.com/link/roadview/"+lat+","+lon+"'
target='_blank'>로드뷰</a>";
                                        content+="</div>";
                                        loadMarker(lat,lon,content);
                                }
                        })
                }

                //마커출력
                let loadMarker = (lat,lon, content) =>{
                        // 지도에 마커를 생성하고 표표시한다
                        var marker = new kakao.maps.Marker({
                            position: new kakao.maps.LatLng(lat,
lon), // 마커의 좌표

                            map: map, // 마커를 표시할 지도 객체
                            clickable:true
                        });
                        var infowindow=new kakao.maps.InfoWindow({
                                content:content,
                                removable:true
                        })
                        //마커 클릭이벤트주기

        kakao.maps.event.addListener(marker,'click',function(){
                                infowindow.open(map,marker);
                        })
```

```
                }
        }
```

지도를 출력해주는 함수를 만들어서 구현하였으며 공공데이터를 가져올 때 서버에 요청하지 않고 바로 가져와 처리할 수 있기 때문에 서블릿을 만들지 않고 구현하였고, 카카오지도를 사용하기 위해 카카오 API를 가져왔다. 현 위치는 navigation.geolocation객체를 이용해서 위도, 경도 값을 가져와서 지도에 이동하고 약국의 위치는 공공데이터에서 제공하는 위도, 경도값으로 marker로 표시하고 상세 내역은 이벤트를 연결하여 클릭시 출력되게 하였다.

공공데이터를 이용하면 다양한 정보를 유용하게 제공할 수 있고 유용한 애플리케이션을 만드는데 많은 도움을 준다.